

OPTIMIZACIÓN HEURÍSTICA Y APLICACIONES

TEMA 3

OPTIMIZACIÓN HEURÍSTICA BASADA EN TÉCNICAS EVOLUTIVAS

Objetivos del Tema:

- ☐ Introducir los fundamentos de los procedimientos basados en búsquedas evolutivas
- ☐ Estudiar los fundamentos de los algoritmos genéticos (GA)
- ☐ Estudiar los fundamentos de las nubes de partículas (PSO)
- ☐ Estudiar los fundamentos de la evolución diferencial (DE)

1 Introducción

2 Algoritmos Genéticos

- 2.1 Fundamentos
- 2.2 Codificación de las soluciones
- 2.3 Funciones y operadores
- 2.4 Temas avanzados

3 Nubes de Partículas

- 3.1 Fundamentos
- 3.2 Funciones y operadores
- 3.3 Temas avanzados

4 Evolución Diferencial

- 4.1 Fundamentos
- 4.2 Funciones y operadores
- 4.3 Temas avanzados

5 Comparativa de las técnicas

1 INTRODUCCIÓN

Tal y como se comentó en el Tema 1, las **metaheurísticas evolutivas** establecen estrategias para dirigir, en el espacio de búsqueda, la evolución de un **conjunto** de posibles soluciones del problema con la intención de acercar los elementos de dicho conjunto a la solución óptima del problema.

Las **características principales** de este tipo de estrategias son las siguientes:

- Exploran el espacio de soluciones utilizando, en cada iteración, un conjunto de soluciones en vez de la única solución que manejan los algoritmos de búsqueda.
- Los mecanismos que modifican iterativamente los valores de cada solución tienen en cuenta los valores del resto de las soluciones de la población y las funciones de ajuste asociadas a las mismas. De esta forma, las soluciones de la población interactúan entre si y evolucionan teniendo en cuenta las interacciones posibles.
- A veces incorporan algún mecanismo adicional, que modifica las soluciones existentes sin tener en cuenta los valores del resto con el objeto de explorar zonas del espacio de búsqueda que no son alcanzables modificando la población con la información contenida en la misma.
- La población actual es sustituida en la iteración siguiente por otro conjunto de soluciones, formado por las nuevas soluciones creadas y en algunos casos parte de las ya existentes. En algunos algoritmos, se utiliza un archivo que almacena los mejores individuos creados hasta el momento, de forma que puedan ser también tenidos en cuenta durante la evolución de las poblaciones.
- Las diferentes metaheurísticas evolutivas se distinguen por la forma en la que combinan la información proporcionada por los elementos de la población.

1 INTRODUCCIÓN

A lo largo de este tema estudiaremos el funcionamiento de tres grandes familias metaheurísticas evolutivas:

- **Algoritmos Genéticos:** son uno de los representantes más populares de las metaheurísticas de evolución, que se inspiraron en los principios de herencia y selección natural de la teoría de evolución de las especies de Darwin. A lo largo de las diferentes iteraciones (generaciones), mantienen una **población**/conjunto de posibles soluciones del problema (individuos de una especie), cuyos valores evolucionan hacia las mejores soluciones del problema (individuos mejor adaptados al medio) mediante un proceso combinado de **selección** de individuos (selección natural) y **operadores genéticos** (cruce y mutación).
- **Nubes de Partículas:** son una metaheurística evolutiva inspirada en el comportamiento social de las bandadas de pájaros o bancos de peces. A lo largo de las diferentes iteraciones, mantienen un conjunto de posibles soluciones del problema (partículas) que son desplazadas a través del espacio de búsqueda teniendo en cuenta su **desplazamiento anterior**, el mejor valor encontrado por la partícula a lo largo de su evolución (el líder o **mejor local**), y el mejor valor encontrado por todas las partículas a lo largo de la evolución (el líder o **mejor global**).
- **Evolución Diferencial:** son una metaheurística evolutiva inspirada en el funcionamiento de algunos métodos de optimización que modifican las soluciones de forma incremental (diferencial), como por ejemplo los métodos de escalada o los de gradiente. A lo largo de las diferentes iteraciones, las técnicas de evolución diferencial mantienen un conjunto de posibles soluciones del problema que son desplazadas incrementalmente teniendo en cuenta **las diferencias existentes entre parejas de soluciones** del conjunto. Por lo tanto, los desplazamientos se adaptan a la diversidad de soluciones existentes en cada iteración.

1 INTRODUCCIÓN

Aunque las diferencias estructurales existentes entre las tres metaheurísticas elegidas dificulta la presentación de una estructura común que represente el comportamiento de todas ellas, es posible establecer algunas analogías entre los mecanismos implementados en las mismas para modificar los valores del conjunto de las soluciones sobre los que están trabajando. Estas analogías no siempre son directas, motivo por el que es conveniente comenzar con el estudio de cada una de las técnicas, y una vez conocida su forma de proceder, comparar la repercusión que los diferentes mecanismos que incorporan tiene en la evolución de las soluciones del algoritmo.

Por lo tanto, este tema se encuentra organizado de la siguiente forma:

1. En las tres primeras secciones se plantea el funcionamiento de cada una de las técnicas por separado. Aunque en principio podrían leerse en cualquier orden, es conveniente hacerlo en el orden elegido, ya que para evitar una repetición innecesaria del material, en el estudio de las dos últimas técnicas se hace referencia a algunos mecanismos que comparten con la primera.
2. En la última sección se comparan la estructura de las tres técnicas y se establecen algunas analogías indirectas entre sus formas de proceder. Para poder entenderlas es necesario entender el funcionamiento de cada técnica por separado, motivo por el que esta sección debe ser estudiada una vez que las otras técnicas han sido desarrolladas.

Por último, es importante aclarar que la mayor dedicación del tema a la primera de las técnicas no significa que sea mejor a las anteriores, si no a que, al ser la más antigua, existen un mayor número de variantes de sus mecanismos básicos que se consideran estándares que conviene conocer. De hecho, la simplicidad de la segunda y tercera técnica propuesta puede ser considerado un plus, en los casos en los que las versiones básicas de estos algoritmos funcionen sobre los problemas que queremos resolver.

La primera familia de técnicas evolutivas que vamos a presentar en este tema son los **algoritmos genéticos** (Genetic Algorithms, GA), nombre bajo el que se engloba a un conjunto de algoritmos inspirados en los principios de herencia y selección natural de la teoría de evolución de las especies de Darwin. Esta teoría defiende que la evolución de una población de individuos de una especie viene determinada por un proceso en el que los individuos mejor adaptados al medio sobreviven y se cruzan entre si para crear los individuos de las nuevas generaciones. La información genética de los hijos es heredada de los padres durante el proceso de cruce, aunque en algunos casos es ligeramente modificada debido a algún mecanismo de mutación.

Los GAs, denominados originalmente “planes reproductivos genéticos” por J. Holland a principios de 1960s, se fundamentan en las ideas anteriormente expuestas:

1. Estableciendo una analogía entre:

- una solución potencial del problema y un individuo de una especie,
- un conjunto de soluciones y una población de individuos,
- el valor de las funciones de mérito sobre cada solución y el grado de adaptación al medio de un individuo.

2. Implementando los mecanismos de selección natural, cruce y mutación, por medio de un conjunto de funciones que en el caso:

- de la selección debe permitir a las soluciones mejores tener mas hijos que las peores,
- del cruce combina los valores de las soluciones de dos individuos diferentes para crear una pareja de hijos,
- de la mutación modifican ocasionalmente los valores de algunas soluciones.

Dicho de otra forma, los GAs utilizan un conjunto (población) de posibles soluciones del problema (individuos de una especie) y evolucionan sus valores hacia la solución óptima del mismo (individuos mejor adaptados al medio) a lo largo de las diferentes iteraciones (generaciones) del algoritmo mediante un proceso combinado de selección de individuos (selección natural) y operadores genéticos (cruce y mutación).

El **Algoritmo 1** muestra el pseudocódigo para la implementación de un GA básico, donde **x** representa a la población (conjunto de soluciones) actual, **parents** al conjunto de parejas de padres seleccionados a partir de la población actual, y **children** a los individuos nuevos creados a partir del cruce de las parejas de padres y del proceso de mutación. Además, la función *Generate()* es la responsable de generar la población inicial, *Select()* elige las parejas de padres teniendo en cuenta los valores de mérito de todas las soluciones de la población actual **x**, *Cross()* y *Mutate()* son los operadores de cruce y mutación, *Recombine()* es la función responsable de elegir entre las soluciones de la población actual **x** y de los hijos recientemente creados **children** aquellas que formarán parte de los individuos de la población en la generación siguiente, y *Stop()* determina cuando se finaliza

Algoritmo 1: GA basico

x=GA(*Generate*, *Select*, *Cross*, *Mutate*, *Recombine*, *Stop*)

```
1  x=Generate()
2  repeat:
3      parents=Select(x)
4      children=Cross(parents)
5      children=Mutate(children)
6      x=Recombine(x, children)
7  until Stop(x)
8  return x
```

la ejecución del algoritmo.

El modo de proceder del algoritmo depende tanto de la representación elegida para codificar las soluciones como del tipo de operaciones implementada en las funciones *Generate()*, *Select()*, *Cross()*, *Mutate()*, *Recombine()* y *Stop()*.

Por último, es importante destacar que las funciones *Select()*, *Recombine()* y *Stop()* habitualmente necesitan conocer el valor de las funciones de mérito del problema.

Una de las características principales que diferencian unos GAs de otros es la forma en la que se codifican las soluciones del problema dentro del algoritmo. La codificación elegida determina el tipo de operadores genéticos (cruce y mutación) que pueden ser utilizados por el algoritmo, y por lo tanto, la forma en la que las soluciones van evolucionando a lo largo de las iteraciones (generaciones) del algoritmo.

Las codificaciones más habituales son las siguientes:

1. **Binaria:** las variables de decisión del problema se representan por medio de una cadena de longitud finita de 0s y 1s.

0	1	1	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---	---	---

La longitud de la cadena viene determinada por el tipo de variable de decisión que se está utilizando:

- Cuando la variable es binaria se puede representar mediante una cadena de longitud 1:

0

- Cuando la variable es entera, y sus valores máximo y mínimo se representan como $\max_v$ y el mínimo como $\min_v$, la longitud de la cadena binaria que nos permite codificar todos los valores enteros posibles es igual a $\log_2(\max_v - \min_v + 1)$. A la hora de evaluar las funciones de mérito se convertirá por medio de un procedimiento auxiliar la cadena binaria a su correspondiente valor entero.

0	1	1	0	0	1
---	---	---	---	---	---

 $\longrightarrow \min_v + 2^4 + 2^3 + 2^0 = \min_v + 25$

- Para representar un variable real con precisión infinita se necesita una cadena binaria de longitud infinita. Como este hecho no es posible, la longitud de la cadena se determina teniendo en cuenta los valores máximo y mínimo de la variable real y la precisión necesaria para representar sus valores. A la hora de evaluar las funciones de mérito se convertirá por medio de un procedimiento auxiliar la cadena binaria a su correspondiente valor real.

0	1	1	0	0	1
---	---	---	---	---	---

 $\longrightarrow \min_v + (2^4 + 2^3 + 2^0) \frac{\max_v - \min_v}{2^6 - 1}$

2. Entera: las variables se representan con cadenas de números enteros.

4	5	1	7	9	8	5	6	0	1
---	---	---	---	---	---	---	---	---	---

La longitud de la cadena depende del tipo de variable de decisión que se está utilizando:

- Cuando la variable es entera se puede representar mediante una cadena de longitud 1:

27

- Cuando la variable es real, la longitud de la cadena se determina teniendo en cuenta los valores máximo y mínimo de la variable real y la precisión necesaria para representar sus valores. A la hora de evaluar las funciones de mérito se convertirá, por medio de un procedimiento auxiliar, la cadena entera a su correspondiente valor real. El siguiente esquema muestra una transformación posible en la que se considera que la cadena representa un número en base 10 con tres cifras decimales (asociada a los tres últimos dígitos).

5	9	0	1	3	8
---	---	---	---	---	---

→
590.138

3. Real: las variables se representan directamente con los tipos de variables que los lenguajes de programación proporcionan directamente para representar números reales (float, double). La precisión, en este caso, viene determinada para la precisión propia del tipo de variable elegida. De hecho, es conveniente tener en cuenta que los tipos de variables reales existentes en los lenguajes de programación codifican los valores reales utilizando unas codificaciones binarias determinadas. La diferencia principal entre el uso de codificación binaria o real en los GAs se encuentra en las operaciones realizadas por los operadores de cruce y mutación. En el caso binario se cruzan y mutan cadenas de números binarios, mientras que en el caso real se cruzan y mutan números reales, a pesar de que éstos sean implementados, a nivel del computador, como números binarios.

4. Codificaciones naturales: por ultimo, en muchos casos es posible codificar las variables de decisión de un problema con una representación directamente relacionada con el problema. Este tipo de codificación recibe el nombre de codificación natural. En el caso de las variables binarias, enteras y reales, la codificación natural es binaria, entera o real. En otros problemas, como la búsqueda de caminos mínimos sobre un grafo, la notación natural es una **permutación** de valores enteros que codifican los nodos del grafo. La existencia de este tipo de codificaciones debe de ser tenido en cuenta siempre y cuando sea posible, ya que la elección de una codificación íntimamente relacionada con el problema puede evitar que los operadores de cruce y mutación, que son los responsables de crear y modificar los valores de las posibles soluciones del problema, generen soluciones no factibles.

En los problemas en los que se optimizan múltiples variables de decisión todas ellas tienen que ser codificadas, creándose de esta forma una cadena de variables binarias, enteras, reales, o de otro tipo, cuyos elementos se encuentran relacionados con las diferentes variables de decisión. A la hora de evaluar las funciones de mérito se dividirá la cadena en las subcadenas asociadas con las diferentes variables de decisión, y a continuación, se convertirán las subcadenas asociadas a cada variable a su valor correspondiente en el espacio de búsqueda.

Además, hay que tener en cuenta que en algunos problemas es posible utilizar la misma representación para todas las variables, y en otros resulta conveniente utilizar representaciones diferentes para cada una de ellas. Por lo tanto, la interpretación de los elementos de la cadena dependerá de la codificación elegida para representar las variables del espacio de búsqueda.

También es importante tener en cuenta que, desde un punto de vista histórico, la primera representación utilizada por J. Holland fue la binaria, y durante muchos años se creyó que ésta era la más adecuada. Esto se debe a que la codificación binaria admite un mayor grado de paralelismo implícito, ya que permite un mayor número de esquemas (o plantillas) que comparten ciertas similitud en algunas posiciones de la cadena a lo largo de toda su longitud. J. Holland también demostró teóricamente que este hecho favorece la diversidad e incrementa la posibilidad de que se formen buenos bloques constructores (o esquemas con un elevado valor en las funciones de mérito) en cada generación.

Sin embargo, la aplicación de otro tipo de codificaciones en múltiples problemas reales ha puesto de manifiesto que el resto de las codificaciones también puede ser exitosa, sobre todo en aquellos casos en los que es necesario determinar el valor óptimo de un elevado número (R) de variables de decisión reales, que codificadas de forma binaria requerirían de una cadena de longitud elevada ($\sum_{i=1:R} B_i$, con B_i el número de bits asociado a la i -ésima variable de decisión) mientras que codificadas de forma real requieren de una longitud más compacta (R).

Por otra parte, con el objeto de acallar las críticas de los detractores de los GAs con codificaciones no binarias, los defensores de los mismos han desarrollado operadores de cruce y mutación que implementan un comportamiento similar al que los operadores de cruce y mutación desarrollados por J. Holland realizan sobre las codificaciones binarias, y que, por lo tanto, permiten extrapolar sus demostraciones teóricas para GAs con representaciones binarias a GAs con representaciones reales.

Por último, es importante resaltar que siempre y cuando la codificación y operadores elegidos permita resolver con éxito un determinado problema, el hecho de carecer de una justificación teórica que explique su funcionamiento no debe hacernos desechar su uso.

Una vez elegida la codificación de las variables de decisión del problema, es necesario determinar las funciones y operadores que determinan el comportamiento del GA básico (representado en el pseudo-código del **Algoritmo 1**):

- *Generate()*: es la función responsable de generar la población inicial del GA. Habitualmente, los valores de las soluciones iniciales son generadas aleatoriamente, aunque también es posible incluir entre los mismos las soluciones obtenidas por cualquier otro algoritmo de optimización.
- *Select()*: es la función que elige las parejas de padres entre las soluciones de la población, teniendo en cuenta su aptitud, es decir, teniendo en cuenta los valores de las funciones de mérito del problema. La probabilidad de elección de cada individuo está relacionado con el valor de sus funciones de mérito, de forma que la información contenida en los individuos mejores se transmite a los nuevos con una frecuencia superior a la información contenida en los peores.
- *Cross()*: es el operador que combina la información contenida en cada pareja de padres para obtener una pareja de hijos. El proceso de combinación también incorpora alguna aleatoriedad, de forma que una misma pareja de padres puede dar lugar a diferentes parejas de hijos.
- *Mutation()*: es el operador que modifica los valores de algunos de los elementos de las cadenas que codifican los valores de algunos individuos de la nueva población. Esta forma de proceder permite al algoritmo explorar nuevas regiones del espacio de búsqueda, que en principio no son alcanzables por el proceso de cruce.
- *Recombine()*: esta función elige los individuos que formarán la población de la generación siguiente entre los individuos de la población actual y los recientemente creados. Según el procedimiento elegido, en esta combinación se tiene o no se tiene en cuenta la aptitud de los individuos existentes.
- *Stop()*: es la función que determina cuándo se termina la ejecución del GA, teniendo en cuenta los valores de la población.

A lo largo de esta sección se presentarán algunas de las funciones y operadores que se utilizan habitualmente en los GAs.

FUNCIÓN DE GENERACIÓN:

Esta función es la **responsable de generar los valores de un conjunto de individuos** de un algoritmo genético. Su funcionalidad se encuentra directamente relacionada con el conocimiento que el usuario tenga sobre la disposición de las soluciones del problema de optimización en el espacio de búsqueda:

- *Si se desconoce* totalmente el valor de las soluciones, lo mas conveniente resulta generar los valores de las soluciones aleatoriamente, siguiendo una distribución uniforme sobre los valores de la codificación elegida. De esta forma, se asegura que todos los puntos del espacio son equiprobables inicialmente.
- *Si se conoce en torno a qué valores* se localizan las soluciones óptimas del problema puede resultar conveniente generar los valores de la codificación elegida siguiendo una distribución Gaussiana en torno a los mismos. Otra forma de proceder consiste en incluir, entre las soluciones propuestas, las mejores soluciones obtenidas por otros algoritmos de optimización, así como algunas generadas aleatoriamente con una distribución uniforme sobre el espacio de búsqueda.

De hecho, cualquier forma de proceder es válida, aunque se debe tener en cuenta que para facilitar la convergencia del GA a la solución óptima del problema es conveniente evitar que el conjunto de sus soluciones converja a un único valor, para que el operador de cruce sea capaz de generar soluciones diferentes. Por lo tanto, la población inicial tiene que ser lo suficientemente diversa para evitar que el GA converja a un valor no óptimo en las generaciones iniciales.

Adicionalmente, y con el objeto de introducir diversidad en la población actual en cualquier generación, la función de generación puede ser utilizada para generar, en cada iteración, un pequeño conjunto de individuos denominados **inmigrantes**, que competirán junto con los individuos generados mediante las operaciones de cruce y mutación, y los ya existentes, para formar parte de la nueva población.

FUNCIÓN DE SELECCIÓN I:

La evolución de los individuos de una población depende directamente de la función que selecciona, teniendo en cuenta los valores de las funciones de mérito del problema, las parejas de padres que serán utilizados por los operadores de cruce para formar las nuevas soluciones.

Esta función, cuya funcionalidad se representa en el pseudo-código de la transparencia, elige N_p parejas de padres entre las N soluciones de la población $\mathbf{x}$, teniendo en cuenta los valores de las funciones de mérito del problema, cuyo cálculo se representa en las líneas 1-3. A continuación, y utilizando alguno de los métodos que veremos en esta sección, se eligen los dos

Función 1: Select genérica

```
parents=Select(x,Np)
1 for i=1:N
2     val(i)=Obj(x(i))
3 end
4 i=1
5 while i<Np
6     parents(i,1)=SelectBasic(x,val)
7     parents(i,2)=SelectBasic(x,val)
8     if valid(parents(i,:))
9         i=i+1;
10    end
11 end
12 return parents
```

padres de cada pareja, y por último, opcionalmente, se determina si la pareja elegida es válida o no. Esta determinación opcional permite desechar parejas formadas por el mismo padre o parejas cuyas soluciones tomen valores muy similares o muy diferentes. Las parejas formadas por el mismo padre no son útiles, ya que los hijos generados en este caso serán igual que el padre repetido. Las parejas con valores de decisión muy similares, habitualmente generan soluciones muy parecidas a las ya existentes, y por lo tanto, concentran la búsqueda en torno a los valores existentes. Las parejas con valores muy diferentes pueden generar soluciones en regiones intermedias del espacio donde las funciones de mérito son peores que las de sus padres, y salirse de los nichos ya identificados por el algoritmo. Por lo tanto, en algunos problemas es conveniente evitar las soluciones muy similares o muy diferentes.

FUNCIÓN DE SELECCIÓN II:

La probabilidad de elección de cada individuo dentro de los métodos de selección básicos se encuentra relacionada con el valor de las funciones de mérito. Según la forma de calcular dicha probabilidad, los métodos existentes se pueden clasificar según diferentes criterios :

- Métodos **estáticos o dinámicos**: en los primeros, la probabilidad permanece constante a lo largo de todas las generaciones, mientras que en los segundos, la probabilidad depende de la generación.
- Métodos **preservativos o extintivos**: en los primeros, todos los individuos tienen alguna probabilidad de ser elegidos, mientras que entre los segundos, algunos individuos tienen probabilidad nula. Entre este segundo grupo, se distingue a su vez entre las técnicas **extintivas con selección izquierda** (en las que se impide que los mejores individuos se reproduzcan para impedir la convergencia prematura del GA), y las técnicas **extintivas con selección derecha** (en la que no se tiene un control específico de la capacidad reproductiva de los individuos más aptos).

La mayoría de los métodos existentes se puede clasificar en dos grandes familias: **los métodos proporcionales y los métodos de torneo**. En los primeros, la selección se realiza teniendo en cuenta la contribución de “aptitud” de cada individuo con respecto al total de la población actual, y por lo tanto, constituyen un tipo de método dinámico. En los segundos, se elige aleatoriamente un subconjunto de soluciones, y a continuación, se selecciona entre las mismas alguna solución en función de los valores de sus funciones de mérito. Fuera de estas dos familias existen algunas técnicas adicionales, como son la *selección de estado uniforme*, *la brecha generacional*, *selección disruptiva*, *jerarquía no lineal* o *selección competitiva*, cuyas propiedades pueden resultar útiles en algunos tipos de problemas.

A continuación, se presentan algunas de las técnicas más habituales pertenecientes a las dos familias anteriormente expuestas.

FUNCIÓN DE SELECCIÓN III – Métodos proporcionales 1:

Los métodos proporcionales, cuyas primeras variantes fueron introducidas por J. Holland, realizan la selección teniendo en cuenta la contribución de “aptitud” de cada individuo con respecto al total de la población actual. La “**aptitud**” de cada individuo se **calcula** teniendo en cuenta los valores de las funciones de mérito:

1. En el caso de los **problemas monoobjetivo**, la aptitud puede estar directamente relacionada con el valor de la función de mérito, o escalada (linealmente o de forma exponencial). En el primer caso, si el problema es de minimización, es conveniente realizar un cambio de signo para que tengan un valor de aptitud más alto aquellas soluciones cuya función de mérito es más pequeño. Además, las aptitudes deben tomar valores positivos en la mayoría de los métodos proporcionales, motivo por el que los valores deben de ser desplazados hacia los valores positivos.
2. En el caso de los **problemas multiobjetivo**, la aptitud tiene que ser calculada con algún procedimiento que asigne un valor escalar a cada uno de las soluciones del problema. Una forma de proceder, que ya hemos visto en temas anteriores, consiste en realizar una combinación lineal de las funciones de mérito. En el caso de querer utilizar la función de Pareto optimalidad, se puede calcular para cada posible pareja de padres qué individuos dominan a otros, y:
 - Asignar un valor de aptitud a los individuos que se encuentran en el primer frente Pareto, otro menor al del segundo frente, otro menor al del tercero ... y así sucesivamente, hasta que todos los individuos estén clasificados.
 - Contabilizar como aptitud de un individuo el número de individuos dominados por éste (de forma que son mejores aquellos que dominan más y peores los que menos).

Nótese que los métodos de escalada también pueden ser aplicados a la hora de calcular la aptitud de los problemas multiobjetivo.

FUNCIÓN DE SELECCIÓN III – Métodos proporcionales 2:

Una vez asignada la aptitud A_i a cada individuo, se puede proceder a seleccionar aquellos que tienen una mejor aptitud utilizando diferentes procedimientos:

- **Método de la ruleta:** es uno de los métodos más simples y populares, implementable a través de las siguientes etapas:

1. Calcular la suma de la aptitud de todos los individuos: $S = \sum A_i$.
2. Generar un número aleatorio r siguiendo la distribución uniforme entre 0 y S .
3. Recorrer los individuos de la población, sumando su aptitud, hasta que la suma sea igual o mayor a r .
4. Elegir como individuo aquel que hace que la suma exceda el valor de r .



Esta forma de proceder requiere que la aptitud de todos los individuos sea positiva, y constituye un método preservativo, en el que el peor individuo puede ser elegido varias veces, aunque la probabilidad de este hecho sea pequeña.

Ejemplo:

Individuo	1	2	3	4	5	6	7	8	9	10
Aptitud	8	2	17	7	2	12	11	7	3	7
Total	8	10	27	34	36	48	59	66	69	76
No. aleatorio			23	49	76	13	1	27	57	
Individuo elegido			3	7	10	3	1	3	7	

FUNCIÓN DE SELECCIÓN III – Métodos proporcionales 3:

- **Sobranter estocástico o muestreo residual:** este método se basa en la idea de calcular, deterministamente, el número de veces que cada individuo i debería ser seleccionado teniendo en cuenta el valor esperado $E_i = \frac{A_i}{\sum_{i=R} A_i} 2N_p$. De esta forma:
 1. Se calcula el número de veces que cada individuo debe ser seleccionado como el entero más próximo, por debajo, a E_i , y se eligen tantos individuos de cada elemento como los que este número determina.
 2. Los valores restantes (r_i , sobranter del redondeo) se usan para rellenar las parejas restantes, hasta conseguir el total de $2N_p$ individuos, utilizando la técnica implementada en el **método de la ruleta**.

Aunque esta forma de proceder hace que la probabilidad de selección se aproxime más a los valores esperados, puede producir una convergencia prematura del GA, ya que establece una mayor presión de selección sobre los valores más aptos.

Ejemplo: considerando que $N_p = 5$

Individuo	1	2	3	4	5	6	7	8	9	10
Aptitud	8	2	17	7	2	12	11	7	3	7
E_i	1.05	0.26	2.24	0.92	0.26	1.58	1.45	0.92	0.39	0.93
N_i	1	0	2	0	0	1	1	0	0	0
r_i	0.05	0.26	0.24	0.92	0.26	0.58	0.45	0.92	0.39	0.93
Total	0.05	0.31	0.55	1.47	1.73	2.31	2.76	3.68	4.07	5

Se elegirán los individuos 1, 3, 3, 6, 7, y los cinco restantes utilizando el método de la ruleta sobre r_i

FUNCIÓN DE SELECCIÓN III – Métodos proporcionales 4:

- **Muestreo determinístico:** es un método similar al sobranter estocástico que requiere un algoritmo de ordenación. Se puede implementar por medio de las siguientes etapas:

1. Calcular la suma de la aptitud de todos los individuos: $S = \sum A_i$
2. Calcular la probabilidad de selección de cada individuo como $P_i = \frac{A_i}{S}$
3. Calcular el valor esperado de ser seleccionado de cada individuo como $E_i = 2N_p P_i$
4. Se calcula el número de veces que cada individuo debe ser seleccionado como el entero más próximo, por debajo, a E_i , y se eligen tantos individuos de cada elemento como los que este número N_i determina.
5. Los valores restantes (sobrantes del redondeo) se ordenan de mayor a menor, y se obtienen las parejas de padres restantes de la parte superior de la lista.

Su efecto es similar al del método anterior: puede producir una convergencia prematura del GA, al ejercer mayor presión sobre los elementos más aptos.

Ejemplo: considerando que $N_p = 5$

Individuo	1	2	3	4	5	6	7	8	9	10
Aptitud	8	2	17	7	2	12	11	7	3	7
E_i	1.05	0.26	2.24	0.92	0.26	1.58	1.45	0.92	0.39	0.93
N_i	1	0	2	0	0	1	1	0	0	0
r_i	0.05	0.26	0.24	0.92	0.26	0.58	0.45	0.92	0.39	0.93
Orden	10	8	9	2	7	4	5	3	6	1

Se elegirán los individuos 1, 3, 3, 6, 7, y los cinco restantes son los 5 primeros de la lista ordenada: 10, 4, 8, 6, 7.

FUNCIÓN DE SELECCIÓN IV – Métodos de torneo 1:

En los métodos de torneo se eligen los padres en dos etapas sucesivas:

1. Se selecciona aleatoriamente, sin tener en cuenta el valor de las funciones de mérito, un subconjunto de tamaño T de las soluciones de la población actual $\mathbf{x}$.
2. Se elige uno de los individuos del subconjunto teniendo en cuenta los valores de sus funciones de mérito.

Según la forma de proceder de la segunda etapa, se distinguen dos métodos de torneo:

- **Torneo determinístico:** Entre todos los individuos del subconjunto elegido en la primera fase, en la segunda se elige el mejor (de acuerdo con la definición de óptimo utilizada en el algoritmo).

Ejemplo: considerando que estamos en un problema de maximización

Individuo	1	2	3	4	5	6	7	8	9	10
Función Merito	8	2	17	7	2	12	11	7	3	7

Se eligen, con $T=4$:
 (2, 3, 4, 8) $\rightarrow$ 3
 (1, 5, 6, 9) $\rightarrow$ 6

FUNCIÓN DE SELECCIÓN IV – Métodos de torneo 2:

- **Torneo Probabilístico:** Utiliza una etapa adicional intermedia para decidir si se elige el individuo mejor o el peor, a partir de un numero r generado aleatoriamente siguiendo una distribución uniforme entre 0 y 1, y de una probabilidad $p > 0.5$ que determina cuando se elige el mejor.

Ejemplo: considerando que estamos en un problema de maximización y que $p = 0.7$

Individuo	1	2	3	4	5	6	7	8	9	10
Función Merito	8	2	17	7	2	12	11	7	3	7

Se eligen, con $T=4$: (2, 3, 4, 8) & $r=0.5 \rightarrow 3$
 (1, 5, 6, 9) & $r=0.7 \rightarrow 5$

Debido a la forma de proceder de ambos métodos, la presión de selección de los individuos mejores se incrementa con el aumento del valor del tamaño de subconjunto, y en el torneo probabilístico se reduce con la disminución del valor de p . Además, el torneo determinístico es extintivo, ya que asigna una probabilidad nula al peor individuo (siempre se elegirá uno mejor que el). Por último, es importante resaltar la simplicidad del método y su independencia del número de funciones de mérito del problema. Aun mas, no es necesario calcular la aptitud de los individuos, únicamente establecer para cada subconjunto elegido en la primera fase, cuales son los mejores (y peores en el torneo probabilístico).

OPERADOR DE CRUCE I:

En los sistemas biológicos, el cruce es un proceso complejo que ocurre entre parejas de cromosomas y que consiste en la alineación de los mismos, y su división en múltiples fragmentos que son combinados para formar los cromosomas del nuevo individuo. Esta forma de proceder se implementa directamente en un conjunto de operadores de cruce de los GAs, que dividen las cadenas de elementos en la que se encuentran codificados los padres y crean los hijos concatenando los fragmentos creados en la división. Dentro de este tipo, que denominaremos **operadores de cruce por puntos**, veremos los operadores de cruce por 1 punto, N puntos, y uniforme.

Para los casos en las que las **variables de decisión** se encuentran **codificadas con valores reales** también se pueden utilizar un conjunto de operadores de cruce adicionales, que *combinan* los valores de cada uno de los elementos de la cadena de los padres para crear los valores de los elementos de cada una de las cadenas de los hijos. Estas combinaciones de valores intentan emular, de alguna forma, la combinación de valores que se produciría si las variables reales estuviesen codificadas de forma binaria, y se aplicasen a cada una de las cadenas asociadas a cada variable un operador de cruce por puntos.

Por otra parte, en los casos en los que la cadena codifica una **permutación** de valores enteros, es conveniente utilizar operadores especiales que aseguren que las cadenas producidas a partir de un cruce siguen siendo permutaciones. Algunos de estos cruces también se presentan en esta sección.

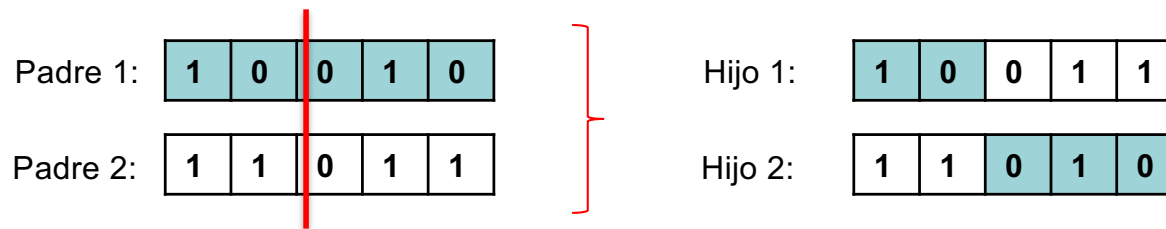
Por último, es importante destacar que aunque en los GAs una pareja de padres produce una pareja de hijos, existen algunas versiones de cruce que desechan uno de los hijos, a costa de perder la información que sus padres le habían transmitido. Además, para asegurar que la información genética de algunos padres se mantiene intacta en la generación siguiente, es posible incluir una etapa previa a la operación de cruce elegido, que determine, a partir de una probabilidad prefijada P_c , si los padres cruzan con el método elegido o pasan a formar parte directamente del conjunto de los hijos.

OPERADOR DE CRUCE II – Cruces por puntos 1:

En este tipo de cruces, las cadenas de variables de los padres son fragmentadas por diferentes puntos, cuyas posiciones se determinan habitualmente de forma aleatoria, y los hijos construidos a partir de la concatenación de las subcadenas resultantes.

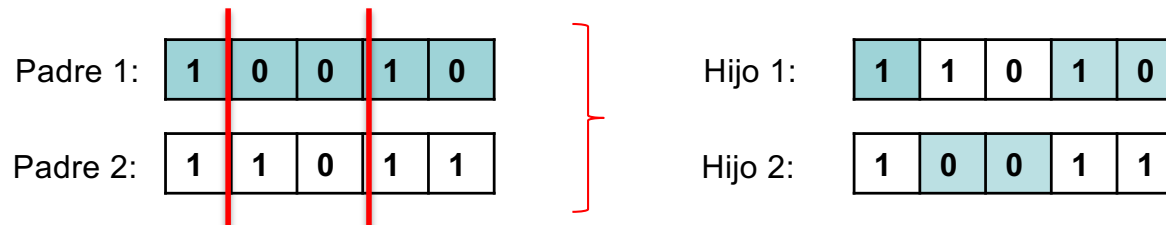
Según la forma de determinar los puntos de cruce, se pueden distinguir los siguientes operadores de cruces por puntos:

- **Cruce por 1 punto:** se determina de forma aleatoria, siguiendo una distribución habitualmente uniforme, el punto de cruce de las dos cadenas, y los hijos se crean concatenando la cadena de un padre con la del otro.



Punto cruce (aleatorio, entre 1 y 4): **2**

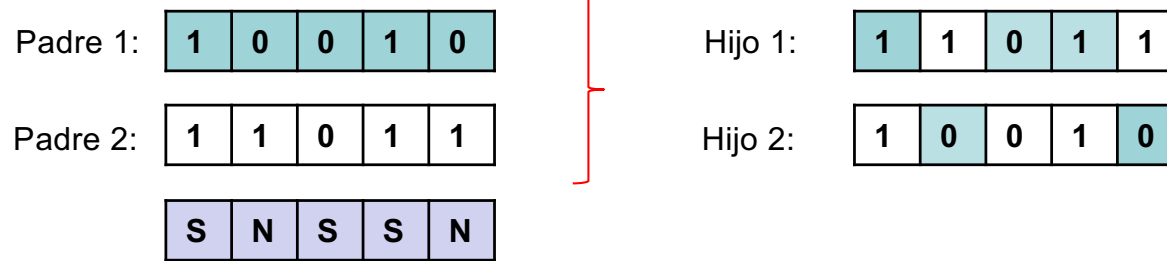
- **Cruce por N puntos:** se determinan de forma aleatoria, siguiendo una distribución habitualmente uniforme, los puntos de cruce de las dos cadenas, y los hijos se crean concatenando, de forma alternada, las cadenas de un padre con las del otro.



2 Puntos de cruce (aleatorios, entre 1 y 4): **1 y 3**

OPERADOR DE CRUCE II – Cruces por puntos 2:

- **Cruce uniforme:** se determina de forma aleatoria, a partir de una probabilidad p , si cada elemento de la cadena del primer padre pertenece al primer hijo o no.



Aleatoriamente se determina que los elementos 1, 3 y 4 se toman del mismo padre y los elementos 2 y 5 del contrario

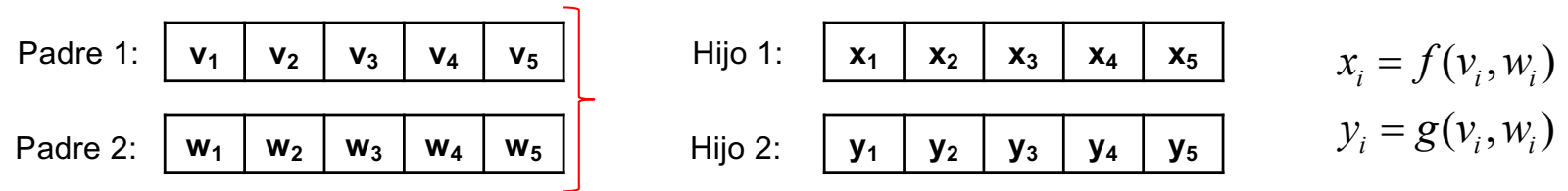
A la hora de elegir el tipo de cruce hay que tener en cuenta que sus características pueden influir en la convergencia del GA. El cruce por 1 punto hace que se mantenga casi intacta la estructura de los padres, y que por lo tanto se mantengan con mayor facilidad los esquemas existentes en la población actual. Los cruces por varios puntos rompen los esquemas con mayor facilidad, motivo por el que no es aconsejable utilizar valores de N elevados. La ruptura de esquemas en el cruce uniforme depende del valor de la probabilidad p : cuando es 0.5 es elevada, y según tiende a sus dos valores extremos disminuye.

En cualquiera de los casos, como el orden en el que aparecen los elementos en la cadena puede también influir en los resultados del algoritmo (ya que valores consecutivos tienen mayor tendencia a formar parte del mismo hijo), existen métodos de cruce por puntos más sofisticados que antes de realizar el cruce reordenan aleatoriamente los elementos de la cadena, acto seguido realizan el cruce, y al final vuelven a reorganizar, siguiendo el orden original, los elementos de la cadena.

Por último, es importante destacar que aunque en los ejemplos mostrados se han representado únicamente cadenas binarias, la metodología presentada se puede utilizar para cadenas con otros tipos de elementos.

OPERADOR DE CRUCE III – Cruces para representaciones reales 1:

Cuando la codificación elegida para representar las variables de decisión del problema es una cadena de variables reales, además de los tres cruces por puntos vistos en la sección anterior, se pueden utilizar otros, en los que, en vez de concatenar los elementos de las subcadenas, se combinan los valores de sus elementos:



Los mas habituales se presentan a continuación:

- **Cruces aritméticos:** esta denominación engloba a un conjunto de cruces fundamentados en la idea de realizar la siguiente combinación lineal de los valores de los padres, en la que el parámetro a_i toma diferentes valores, según sea el tipo de cruce implementado:

$$x_i = a_i v_i + (1 - a_i) w_i$$

$$y_i = a_i w_i + (1 - a_i) v_i$$

- *Cruce aritmético total:* En este caso, todos los valores de la cadena son combinados utilizando el mismo valor de a , que se elige aleatoriamente según una distribución uniforme entre 0 y 1, para todos los elementos de la cadena.

$$a \sim U(0,1)$$

$$a_i = a$$

<i>Aleatoriamente</i> $a = 0.2$	Hijo 1:	$0.2v_1+0.8w_1$	$0.2v_2+0.8w_2$	$0.2v_3+0.8w_3$	$0.2v_4+0.8w_4$	$0.2v_5+0.8w_5$
	Hijo 2:	$0.8v_1+0.2w_1$	$0.8v_2+0.2w_2$	$0.8v_3+0.2w_3$	$0.8v_4+0.2w_4$	$0.8v_5+0.2w_5$

OPERADOR DE CRUCE III – Cruces para representaciones reales 2:

- Cruce intermedio:** en este caso, se elige aleatoriamente un punto de cruce k a partir del cual se combinan los valores de los padres, y un valor fijo a entre 0 y 1 para todos los valores que se combinan.

$$k \sim U(\text{PuntosCruce})$$

$$a \sim U(0,1)$$

$$a_i = \begin{cases} 1 & i \leq k \\ a & \text{c.c.} \end{cases}$$

Aleatoriamente
 $k = 2, a = 0.2$

Hijo 1:	v_1	v_2	$0.2v_3+0.8w_3$	$0.2v_4+0.8w_4$	$0.2v_5+0.8w_5$
Hijo 2:	w_1	w_2	$0.8v_3+0.2w_3$	$0.8v_4+0.2w_4$	$0.8v_5+0.2w_5$

- Cruce aritmético simple:** Se elige un punto de cruce como en el caso anterior, pero se muestrea un valor diferente para cada a_i dentro de un rango de valores que tiene en cuenta los límites inferiores (l_i) y superiores (u_i) para cada variable del problema y los valores que se están combinando.

$$k \sim U(\text{PuntosCruce})$$

$$\alpha_i = \frac{l_i - w_i}{v_i - w_i}, \beta_i = \frac{u_i - w_i}{w_i - v_i}, \gamma_i = \frac{l_i - v_i}{w_i - v_i}, \delta_i = \frac{u_i - w_i}{v_i - w_i}$$

$$range_k = \begin{cases} [\max(\alpha_i, \beta_i), \min(\gamma_i, \delta_i)] & \text{si } v_i > w_i \\ [0, 0] & \text{si } v_i = w_i \\ [\max(\gamma_i, \delta_i), \min(\alpha_i, \beta_i)] & \text{si } v_i < w_i \end{cases}$$

$$b_i \sim U(range_i)$$

$$a_i = \begin{cases} 1 & i \leq k \\ b_i & \text{c.c.} \end{cases}$$

OPERADOR DE CRUCE III – Cruces para representaciones reales 3:

- **Cruce binario simulado** (Simulated Binary Crossing, SBX): este método intenta emular directamente, sobre los valores reales, el cruce de 1 punto usado sobre la representación binaria de la variable real correspondiente. Para implementarlo, se realizan las siguientes etapas para cada una de las variables reales de la cadena:

1. Se genera un número aleatorio u_i , distribuido uniformemente en el intervalo $[0,1]$.
2. Se calcula el valor de β_i en función del valor de u_i y de un parámetro η_c ($= 1$ ó 2).
3. Se realiza el cruce con la siguiente expresión:

$$x_i = 0.5((v_i + w_i) - \beta_i(w_i - v_i))$$

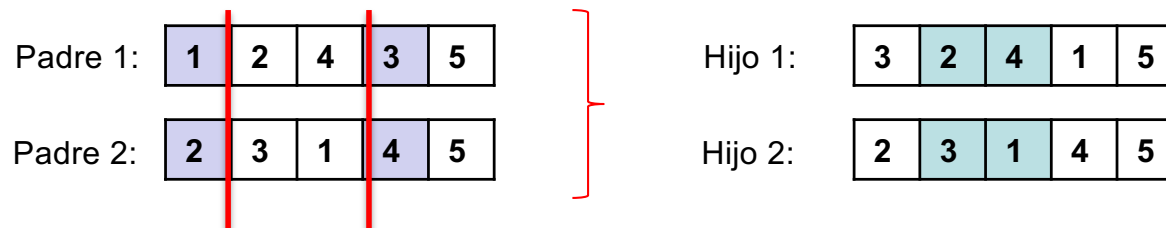
$$y_i = 0.5((v_i + w_i) + \beta_i(w_i - v_i))$$

$$\beta_i = \begin{cases} (2u_i)^{\frac{1}{\eta_c+1}} & \text{si } u_i < 0.5 \\ \left(\frac{1}{2(1-u_i)}\right)^{\frac{1}{\eta_c+1}} & \text{c.c.} \end{cases}$$

OPERADOR DE CRUCE IV– Cruces para permutaciones 1:

En el caso de que la cadena de valores recoja una permutación de números enteros, es conveniente utilizar un conjunto de cruces especiales que aseguran que los hijos creados al combinar los valores de los padres constituyen en si mismos una permutación valida de valores.

- **Cruce ordenado** (*Order Crossover, OX*): en este tipo de cruce se mantiene la organización de una parte de la cadena de cada padre, y el resto de los valores se completa siguiendo el orden de valores establecidos en el otro padre. La forma de proceder es la siguiente:
 1. Seleccionar aleatoriamente una subcadena de los padres.
 2. Reproducir en los hijos la subcadena elegida en las posiciones correspondientes.
 3. Completar el resto de las posiciones utilizando los valores que se encuentran en el otro padre, siguiendo el orden en el que el aparecen, sin tener en cuenta los valores que ya ha heredado en el paso 2.

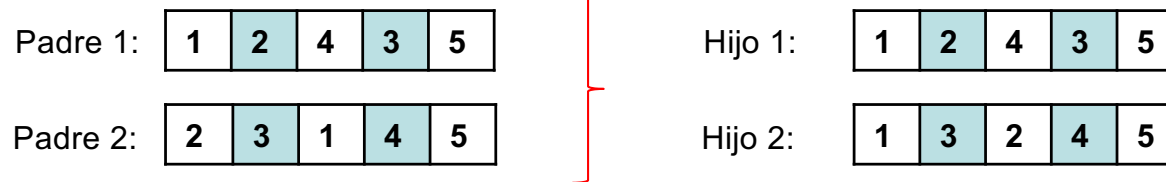


Subcadena elegida aleatoriamente, entre los dos puntos de cruce marcados en rojo. En violeta, marcamos los elementos que tenemos que eliminar para completar los valores.

OPERADOR DE CRUCE IV– Cruces para permutaciones 2:

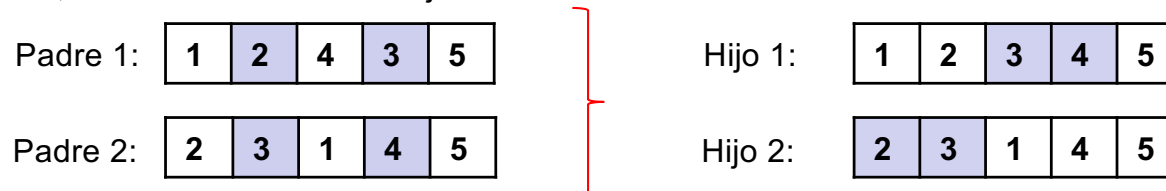
- **Cruce basado en posición** (*Position-based Crossover*): es similar al anterior, pero en vez de elegir un conjunto consecutivo de valores, se eligen un conjunto finito de valores. La forma de proceder es la siguiente:

1. Seleccionar aleatoriamente algunos elementos de la cadena.
2. Reproducir en los hijos los elementos elegidos en las posiciones correspondientes.
3. Completar el resto de las posiciones utilizando los valores que se encuentran en el otro padre, siguiendo el orden en el que aparecen, sin tener en cuenta los valores que ya ha heredado en el paso 2.



Los elementos elegidos aleatoriamente se marcan en verde y se llevan a los hijos. Los restantes se completan con los del otro padre, manteniendo el orden, sin repetir los que ya han sido completados.

- **Cruce basado en orden** (*Order-based Crossover*): variación del anterior, en el que los valores elegidos en el apartado 1, en vez de ser utilizados, son eliminados de los hijos del contrario.



Cambia el orden de los eliminados

No cambia el orden de los eliminados por que coinciden

Los elementos elegidos aleatoriamente se marcan en violeta, y se eliminan del hijo del otro padre. Los que no son eliminados, se llevan a los hijos directamente (en blanco) y se completa con los eliminados en el orden en el que éstos aparecían.

OPERADOR DE MUTACIÓN I:

El operador de mutación, que se aplica a los hijos obtenidos después del cruce, es considerado un operador secundario en muchos GA, ya que sólo se aplica a un porcentaje pequeño de los elementos de las cadenas de valores. A pesar de este carácter secundario, es un elemento importante dentro del GA, ya que permite alcanzar regiones del espacio de búsqueda que no son alcanzables a través de la concatenación o combinación de los valores de los individuos existentes en la población actual. Es decir, es el responsable de mover la búsqueda del GA a otras regiones del espacio, y de evitar la convergencia prematura de todos los individuos de una población a un único valor.

Para las codificaciones binarias, enteras y reales, podemos distinguir dos etapas en su funcionamiento:

1. Se calculan las posiciones de las cadenas cuyos valores van a ser mutados, habitualmente utilizando una función aleatoria que determina, a través de una probabilidad de mutación P_m , si cada punto de la cadena tiene que ser mutado o no.

Etapa 1: Variable aleatoria sobre la que se decide si hay mutación $m_i \sim U(0,1)$

$Muta_i$ si $m_i < P_m$ ($P_m = 0.02$)

4	2	1	1	0
0.01	0.8	0.3	0.6	0.5
S	N	N	N	N

Para evitar que se realicen muchas mutaciones, es conveniente elegir valores bajos de la probabilidad de mutación. Mientras algunos autores recomendaban inicialmente valores fijos e independientes de la longitud de la cadena (un P_m perteneciente a $[0.001, 0.01]$), en la actualidad se considera un valor apropiado el inverso de la longitud de la cadena. Por otra parte, algunos GA utilizan probabilidades de mutación elevadas al principio, y poco a poco, las van reduciendo.

OPERADOR DE MUTACIÓN II:

2. Se calculan los nuevos valores asociados a la posiciones de la cadena que deben mutar:

- En el caso binario, se elige el valor contrario al que tienen.
- En el caso entero, cualquier valor del rango valido de los valores enteros.
- Y en el caso real, se eligen los valores siguiendo alguna de las estrategias que veremos a continuación.

Valor inicial del individuo

4	2	1	1	0
---	---	---	---	---

Etapla 1: Variable aleatoria sobre la que se decide si hay mutación

$$m_i \sim U(0,1)$$

$$Muta_i \text{ si } m_i < P_m \quad (P_m = 0.02)$$

0.01	0.8	0.3	0.6	0.5
------	-----	-----	-----	-----

S	N	N	N	N
---	---	---	---	---

Etapla 2: Se elige un nuevo valor, dentro del rango valido de enteros: $V \sim U(\text{rango})$

3	2	1	1	0
---	---	---	---	---

Por otra parte, existen operadores especialmente diseñados para las permutaciones que obtienen como resultado directamente permutaciones validas. Algunos de los mismos también serán presentados en esta sección.

OPERADOR DE MUTACIÓN III – Mutaciones para representaciones reales 1

Cuando se utiliza una representación real, los valores a los que mutan las posiciones de la cadena que tienen que cambiar su valor, se pueden calcular con diferentes procedimientos:

- **Mutación Uniforme:** el nuevo valor del elemento que muta se elige utilizando una distribución uniforme entre el valor máximo (u_i) y mínimo (l_i) de ese parámetro. Es la mutación real más sencilla, y permite alcanzar cualquier región del espacio de búsqueda con la misma probabilidad.

Valor inicial del individuo

Etapa 1: $Muta_i$

Etapa 2: $w_i \sim U(l_i, u_i)$

v_1	v_2	v_3	v_4	v_5
-------	-------	-------	-------	-------

S	N	N	N	N
---	---	---	---	---

w_1	v_2	v_3	v_4	v_5
-------	-------	-------	-------	-------

- **Mutación No Uniforme:** se calcula el nuevo valor del elemento que muta teniendo en cuenta su valor actual, los valores de los límites, la generación actual t , y la final T . La forma de proceder, que se muestra en las ecuaciones de la derecha, hace que la perturbación $\Delta()$ introducida sobre el valor actual pueda ser mayor en los instantes iniciales del algoritmo, y pequeña en los instantes finales. El tamaño y sentido de la perturbación se elige aleatoriamente a través de las variables r_i y s_i . Por último, el parámetro b define el grado de no uniformidad de la mutación (habitualmente toma valores en torno a 5).

$$r_i \sim U(0,1)$$

$$s_i \sim U(0,1)$$

$$w_i = \begin{cases} v_i + \Delta(t, u_i - v_i) & \text{si } r_i < 0.5 \\ v_i - \Delta(t, v_i - l_i) & \text{c.c.} \end{cases}$$

$$\Delta(t, r) = y \left(1 - s^{\left(1 - \frac{t}{T}\right)^b} \right)$$

OPERADOR DE MUTACIÓN III – Mutaciones para representaciones reales 2

- **Mutación al Limite:** el nuevo valor del elemento que muta se elige aleatoriamente entre uno de los dos limites existentes.

Valor inicial del individuo

v_1	v_2	v_3	v_4	v_5
-------	-------	-------	-------	-------

Etapa 1: $Muta_i$

S	N	N	N	N
---	---	---	---	---

$r_i \sim U(0,1)$

Etapa 2:

$$w_i = \begin{cases} l_i & \text{si } r_i < 0.5 \\ u_i & \text{c.c.} \end{cases}$$

w_1	v_2	v_3	v_4	v_5
-------	-------	-------	-------	-------

- **Mutación basada en polinomios** (*Polynomial-Based Mutation*): este tipo de mutación intenta emular directamente, sobre los valores reales, el comportamiento de la mutación sobre la representación binaria de la variable real correspondiente. Para implementarlo, habitualmente después de un cruce binario simulado, se pueden realizar las siguientes operaciones:

1. Se genera un numero aleatorio u_i , distribuido uniformemente en el intervalo $[0,1]$.
2. Calcular el valor de δ_i en función del valor de u_i y de un parámetro η_m
3. El valor de la posición mutada se calcula como: $w_i = v_i + (u_i - l_i) \delta_i$

$$\delta_i = \begin{cases} (2u_i)^{\frac{1}{\eta_m+1}} - 1 & \text{si } u_i < 0.5 \\ 1 - (2(1-u_i))^{\frac{1}{\eta_m+1}} & \text{c.c.} \end{cases}$$

Para modificar el comportamiento de esta mutación con el paso de las generaciones t , se sugiere usar un valor de $\eta_m = 100+t$.

OPERADOR DE MUTACIÓN IV – Mutaciones para permutaciones 1

En los operadores de mutación especialmente diseñados para las permutaciones no aparecen las dos etapas vistas en los operadores de mutación anteriores, ya que su funcionamiento se basa en la idea de intercambiar los valores que se encuentran en unas posiciones de la cadena por los valores que se encuentran en otras, de forma que la cadena resultante siga siendo una perturbación válida.

Entre las propuestas existentes, destacaremos las siguientes:

- **Mutación por inserción:** se selecciona un valor arbitrario, y se inserta en otra posición arbitraria de la cadena. Si el proceso se repite con varios elementos se produce la **mutación por desplazamiento**.

Valor inicial del individuo

4	2	5	3	1
---	---	---	---	---

Se eligen aleatoriamente los valores 2 y 1 y las posiciones 1 y 4:

Primer cambio ($2 \rightarrow Pos_1$):

2	4	5	3	1
---	---	---	---	---

Segundo cambio ($1 \rightarrow Pos_4$):

2	4	5	1	3
---	---	---	---	---

- **Mutación por intercambio recíproco:** se seleccionan dos puntos de la cadena al azar y se intercambian sus valores.

Valor inicial del individuo

4	2	5	3	1
---	---	---	---	---

Se eligen aleatoriamente las posiciones 2 y 3:

4	5	2	3	1
---	---	---	---	---

OPERADOR DE MUTACIÓN IV – Mutaciones para permutaciones 2

- **Mutación heurística:** se selecciona un subconjunto de los genes de la cadena al azar, y con los mismos, se crean tantos individuos como permutaciones de esos genes sean posibles. Entre todos los nuevos individuos creados se elige el mejor, de acuerdo con las funciones de mérito y la definición de optimalidad utilizada. Esta última etapa, de elección del mejor, no es habitual en las mutaciones estándar, y aunque en principio favorece la creación de individuos mejores en el proceso de mutación, puede dificultar la convergencia hacia el valor óptimo en algunos problemas.

Valor inicial del individuo

4	2	5	3	1
---	---	---	---	---

Se eligen aleatoriamente las posiciones 2, 3 y 5. Permutando sus valores se obtienen 6 posibles soluciones

4	2	5	3	1
---	---	---	---	---

4	2	1	3	5
---	---	---	---	---

4	5	2	3	1
---	---	---	---	---

4	5	1	3	2
---	---	---	---	---

4	1	5	3	2
---	---	---	---	---

4	1	2	3	5
---	---	---	---	---

La solución mejor, de acuerdo con las funciones de mérito, es la que se considera como individuo mutado

4	5	1	3	2
---	---	---	---	---

FUNCIÓN DE RECOMBINACIÓN I

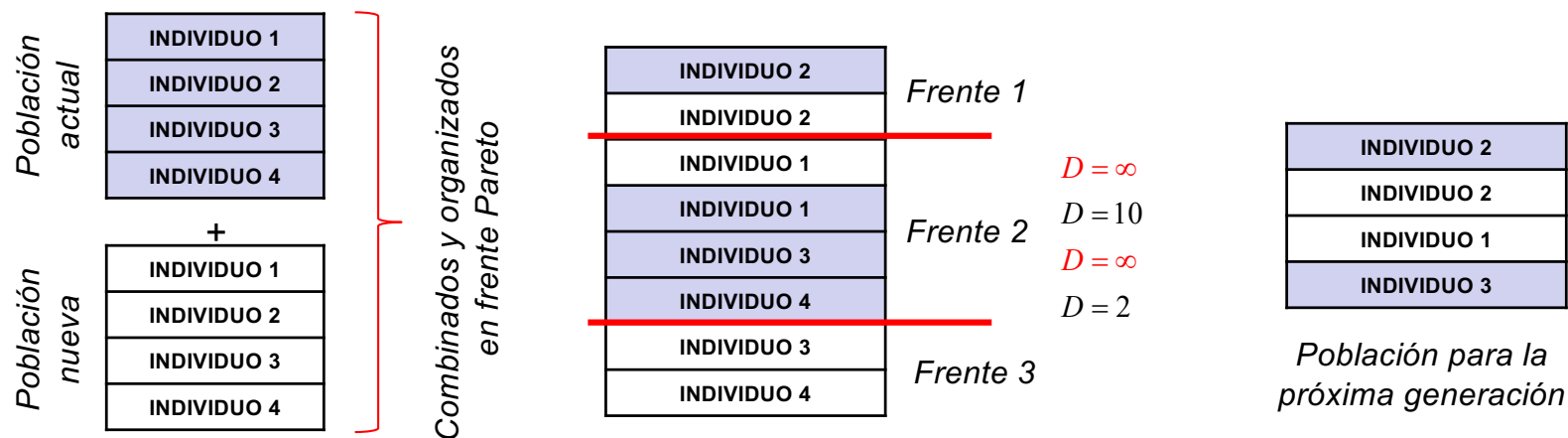
Una vez que los operadores de cruce y mutación han creado los nuevos individuos de la población, es necesario decidir cuales de los antiguos, o de los nuevos, formarán parte de la población en la próxima generación. El número de funciones de recombinación es elevado, y da lugar a una gran variedad de GAs. A continuación, introduciremos algunas de las más sencillas que se pueden implementar, y una de las presentes en los GAs multiobjetivo mas habituales.

- **Sustitución generacional:** Es el método más sencillo, ya que lo único que hay que hacer es sustituir la población actual por la nueva. Para conseguir que el tamaño de la población permanezca constante, es necesario crear tantos hijos como individuos existen en la población. Por otra parte, como todos los individuos existentes desaparecen, esta forma de proceder puede hacer que se pierda la mejor solución encontrada hasta el momento. Para evitarlo, la mejor solución obtenida a lo largo de todas las generaciones puede ser almacenada en un archivo auxiliar. En el caso en el que se trate de un problema multiobjetivo y una optimización del tipo Pareto, el número de individuos en el frente Pareto obtenido por el algoritmo no siempre es el mismo. En algunos casos, es necesario limitar el tamaño del archivo, pudiéndose usar métodos que tengan en cuenta la dispersión de los elementos del frente.
- **Elitismo:** Con el objeto de asegurar que la población mantiene la mejor solución obtenida por el GA a lo largo de todas las generaciones, se puede forzar que la mejor solución (o el conjunto de soluciones mejores perteneciente al frente Pareto) de la población actual, pase a formar parte de la población en la próxima generación. El resto de los individuos que la constituyen pueden ser elegidos entre los nuevos, o entre los nuevos y los antiguos. Esta forma de proceder, denominada elitista porque asegura la supervivencia de los mejores, puede hacer que el tamaño de la población crezca si el número de soluciones que forman el frente Pareto crece a lo largo de las iteraciones. En algunos algoritmos mutliobjetivo se limita el tamaño de la población teniendo en cuenta la dispersión de los elementos del frente.

FUNCIÓN DE RECOMBINACIÓN II – NSGA-II 1

A continuación, vamos a presentar el método de recombinación del **NSGA-II**, que es uno de los GAs considerados estándar para problemas multiobjetivo. La forma de proceder, para asegurar que la población de la generación siguiente tiene el mismo tamaño que la población (N_p) de la generación actual, es la siguiente:

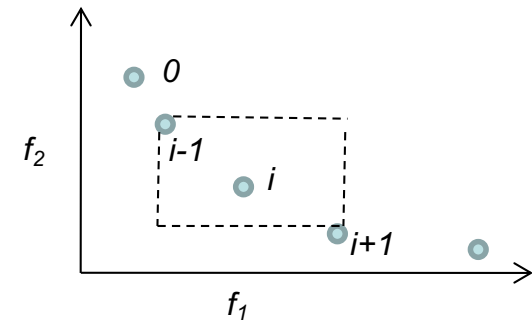
1. Se crea una población formada por todos los elementos de la población actual y los nuevos.
2. Se utiliza la definición Pareto para clasificar los individuos de la población total en frentes sucesivos: en el primer frente se encuentran todos los individuos que no están dominados por otros, en el segundo todos los que únicamente están dominados por individuos del primer frente, en el tercero los que están dominados por los del primero y el segundo ... y así sucesivamente.
3. Una vez organizados, se eligen los N_p primeros elementos, para determinar el frente Pareto en el que aparece el último de ellos. Todos los individuos de los frentes anteriores forman parte de la población de la nueva generación. Para seleccionar los individuos del frente determinado que forman parte de la población en la próxima generación, se calcula la “crowding distance” de todos los individuos que constituyen el frente, y se seleccionan aquellos con mayor *crowding distance*.



FUNCIÓN DE RECOMBINACIÓN II – NSGA-II 2

La “crowding distance” de un conjunto de individuos (en el caso del NSGA-II, aquellos que se encuentran dentro del frente Pareto que queremos dividir) se obtiene:

1. Calculando los valores de las funciones de mérito de todos los individuos.
2. Para cada función de mérito, se organizan de menor a mayor sus valores, y se calcula la distancia asociada a cada individuo y función de mérito como la separación entre el valor inmediatamente superior e inferior dividido entre la diferencia entre el valor máximo y mínimo. Para los valores de los extremos, el de la distancia asociada a dicha función de mérito es infinito.
3. Una vez calculadas las componentes asociadas a todas las funciones de mérito e individuo, se calcula la *crowding distance* sumando, para cada individuo, las componentes asociadas a todas las funciones de mérito.



Los elementos con *crowding distance* igual a infinito resultan ser valores extremos del frente elegido, por lo que NSGA-II los elige como representantes extremos del frente. Del resto, aquellos que tienen valores mayores representan puntos en los que la dispersión es mayor, motivo por el que NSGA-II los prefiere a aquellos que tienen una distancia menor, para cubrir el frente con un número limitado de puntos.

FUNCIÓN DE PARADA

Por último, para terminar la ejecución del GA, es necesario definir la función de finalización del algoritmo. Las versiones más sencillas y habituales consisten en:

- **Límite de generaciones:** el algoritmo termina una vez que se alcanza un número de generaciones prefijado.
- **Convergencia de la población:** el algoritmo también puede terminar cuando la diferencia de los valores de la cadena o de las funciones de mérito de la población es menor que un valor prefijado, o cuando las mejores soluciones o sus funciones de mérito no cambian de forma significativa durante un número prefijado de generaciones. La primera forma de proceder mide la convergencia de todos los individuos a una región del espacio, mientras que la segunda mide la convergencia de los mejores hacia un valor determinado.

En cualquier caso, una vez terminada la ejecución del algoritmo, éste devuelve los individuos de toda la población en la última generación. Para saber qué soluciones son las mejores, habrá que calcular los valores de las funciones de mérito, y ordenar las soluciones teniendo en cuenta el criterio de optimalidad elegido. De hecho, como el cálculo de los valores de las funciones de mérito es necesario en las funciones de selección y recombinación, y opcionalmente en las de parada, puede ser conveniente calcular sus valores en el bucle principal del **Algoritmo 1**, y hacer que formen parte de los parámetros de entrada de las funciones *Select()*, *Recombine()* y *Stop()*, y de salida de *Recombine()*. De esta forma, los valores de las funciones de mérito no sólo estarán disponibles para que lo devuelva el GA, si no que no será necesario recalcularlos dentro de las tres funciones anteriores.

Optimización de problemas multi-objetivo 1

Para convertir un GA en una técnica de optimización multiobjetivo es suficiente con utilizar *funciones de selección y recombinación* que sean capaces de elegir las parejas de padre o los individuos que formarán parte de la próxima generación teniendo en cuenta los valores de múltiples funciones de mérito a la vez. Aunque una de las soluciones más sencillas consiste en convertir el problema multiobjetivo en uno monoobjetivo combinando el valor de los objetivos en un único valor, en muchos problemas resulta conveniente utilizar directamente las definiciones de optimalidad Pareto en las funciones de selección y recombinación. Aunque en las secciones correspondientes a ambas funciones ya se especificó el tratamiento que se puede dar a los problemas multiobjetivo dentro de las mismas es importante recordar que:

- Entre las dos familias presentadas de **funciones de selección**, las funciones de *torneo* son mucho *más sencillas* de usar para los problemas multiobjetivo ya que en este caso, el uso directo de la definición de optimalidad Pareto permite determinar qué individuos son seleccionados para formar parejas de padres. Para poder utilizar una función de selección proporcional es necesario asignar un único valor de aptitud a cada individuo, pudiéndose usar para este fin un valor asociado a la inversa del número (primero, segundo, tercero ...) de frente Pareto en el que se encuentra o la cuenta del número de veces que cada solución es dominada por el resto.
- Cuando se utiliza una función de **recombinación elitista** todas las soluciones que se encuentran en cada frente Pareto son igualmente buenas, motivo por el que 1) si se quieren mantener todas ellas el tamaño de la población variará y 2) si se quiere mantener el tamaño de la población constante será necesario seleccionar, utilizando algún criterio que permita obtener muestras distribuidas a lo largo de todo el frente Pareto (por ejemplo el del NSGA-II)
- Si se utiliza una **recombinación generacional**, y se quiere mantener el óptimo, los valores ya localizados del frente Pareto han de ser almacenados en un archivo de 1) tamaño variable o 2) fijo, tras un muestreo adecuado de los valores del frente.

Optimización de problemas multi-objetivo 2

La forma de tener en cuenta los puntos anteriormente expuestos a la hora de diseñar GAs multiobjetivos no es única y ha dado lugar a un amplio conjunto de algoritmos, algunos de los cuales son considerados en la actualidad GAs multiobjetivo estándar .

Entre los más importantes, podemos destacar los siguientes:

- **MOGA** (Multiple Objective GA): C.M. Fonseca, and P.J. Fleming. Genetic Algorithms for Multiobjective Optimization: Formulation, Discussion and Generalization. Fifth International Conference in Genetic Algorithms. 1993.
- **SPEA-II** (Strength Pareto Evolutionary Algorithm): E. Zitzler, M. Laumann, L. Thiele. SPEA2: Improving Strength Pareto Evolutionary Algorithm. EUROGEN 2001.
- **NSGA-II** (Nondominated Sorting GA): K. Deb, A. Pratap, S. Agarwal, T. Meyarivan. A Fast and Elistist Multiobjective Genetic Algorithm: NSGA-II. IEEE Transactions on Evolutionary Algorithms. 2002.
- **NPGA-II** (Niched-Pareto GA): M. Erickson, A. Mayer, J. Horn. The Niched Pareto Genetic Algorithm 2 Applied to the Design of Groundwater Remediation Systems. First International Conference on Evolutionary Multi-Criterion Optimizationn. 2001.
- **PAES** (Pareto Archived ES): J.D. Knowles, D.W. Corne. Approximation the Nondominated Front Using Pareto Archived Evolution Strategy. Evolutionary Computation. 2000.
- **PESA** (Pareto Envelope-based Selection Algorithm): D.W. Corne, N.R. Jerram, J.D. Knowles, M.J. Oates. PESA-II: Region-based Selection in Evolutionary Mutliobjective Optimization. GECCO 2001.

Tratamiento de soluciones no factibles

Los operadores de cruce y mutación que hemos visto en las secciones anteriores son capaces de crear cadenas de números binarios, enteros o reales entre dos valores determinados, o cadenas de enteros que constituyen una permutación de valores.

En los casos en los que las restricciones del problema que se desee resolver requieran de otro tipo de cadenas de valores, el GA debe de resolver el problema de la infactibilidad de las soluciones con algún mecanismo adicional:

- Utilizando alguna función de mérito, capaz de determinar cuándo las soluciones son factibles y cuándo no, y utilizando los valores obtenidos por dicha función de mérito a la hora de decidir qué soluciones son las mejores o calcular su aptitud. Esta forma de proceder es habitualmente la más sencilla, aunque hay que tener en cuenta que convierte los problemas mono-objetivo en problemas multiobjetivo.
- Utilizando una función adicional de reparación después de la mutación que modifique convenientemente los valores de las soluciones no factibles de forma que éstas se conviertan en soluciones factibles. Esta opción es mas complicada que la anterior en la mayoría de los problemas, y al rehacer las estructuras creadas por los operadores de cruce y mutación, invalida parte del trabajo que éstos realizan. En los casos en los que las funciones de reparación tengan que modificar los valores de una gran parte de los individuos creados, será conveniente analizar los mecanismos que hacen que los operadores de cruce y mutación creen tantas soluciones no factibles.
- Diseñar nuevos operadores de cruce y mutación, especialmente diseñados para el problema que se desee resolver, que aseguren, por diseño, la factibilidad de las soluciones que generan. Un ejemplo de esta forma de proceder la constituyen los operadores especialmente diseñados por las permutaciones. Aunque esta aproximación puede resultar más complicada que las restantes, siempre debe de ser tomada en cuenta, ya que la creación de operadores específicos para un problema determinado también permite codificar en los mismos el conocimiento que se tiene del problema y facilitar la labor del GA.

La segunda familia de técnicas evolutivas que vamos a presentar en este tema son las **nubes de partículas** (*Particles Swarm Optimization*), nombre bajo el que se engloba a un conjunto de algoritmos inspirados en los movimientos coreográficos de una bandada de pájaros mientras están buscando alimentos. El comportamiento social de la bandada se basa en un intercambio de información entre los pájaros cuando localizan alimentos, y en una exploración individual del espacio de búsqueda en el que se tiene en cuenta su trayectoria pasada y la información recibida del resto de los pájaros.

Los PSOs, originariamente propuestos por J. Kennedy y R. Eberhart en 1995, se fundamentan en la búsqueda distribuida y coordinada de la bandada de pájaros:

1. Estableciendo una analogía entre:

- una solución potencial del problema y la posición de un individuo (partícula) de la bandada,
- un conjunto de soluciones y las posiciones de una bandada (o nube) de individuos,
- el valor de las funciones de mérito sobre cada solución y la concentración de alimentos en un punto del espacio.

2. Resolviendo las ecuaciones dinámicas que determinan el movimiento de la bandada de pájaros, utilizando unas sencillas ecuaciones en diferencias que tienen en cuenta:

- la dinámica previa de cada partícula,
- la mejor posición observada por cada partícula a lo largo de su trayectoria,
- la mejor posición observada por todas las partículas a lo largo de sus trayectorias.

Dicho de otra forma, los PSOs desplazan a un conjunto (nube) de posibles soluciones del problema (partículas) a través del espacio de búsqueda teniendo en cuenta su **desplazamiento anterior**, el mejor valor encontrado por la partícula a lo largo de su evolución (el líder o **mejor local**), y el mejor valor encontrado por toda la nube a lo largo de la evolución (el líder o **mejor global**).

El **Algoritmo 2** muestra el pseudocódigo para la implementación de un PSO básico, donde $\mathbf{x}$ representa a la posición actual de la nube (conjunto de soluciones), $\mathbf{v}$ su velocidad, **lbest** (*local best*) la mejor posición que cada una de las partículas de la nube ha encontrado a lo largo de su trayectoria, y **gbest** (*global best*) la mejor posición que todas las partículas de la nube han encontrado a lo largo de todas sus trayectorias. La función *Initialize()* es la responsable de generar la posición y velocidad inicial de todos los elementos de la nube, *Best()* se encarga de determinar las posiciones locales y globales mejores a lo largo de la trayectoria, *Dynamic()* actualiza la posición y velocidad de todas las partículas, y *Stop()* determina cuándo se finaliza la ejecución del algoritmo.

Algoritmo 2: PSO básico

```
gbest=PSO()  
1  [x,v]=Initialize()  
2  [lbest,gbest]=Best(x)  
3  repeat:  
4      [x,v]=Dynamic(x,v,lbest,gbest)  
5      [lbest,gbest]=Best(x,lbest,gbest)  
6  until Stop(gbest)  
7  return gbest
```

El modo de proceder del algoritmo depende tanto de la representación elegida para codificar las soluciones como del tipo de operaciones implementada en las funciones *Initialize()*, *Best()*, *Dynamic()* y *Stop()*. Aunque en la actualidad se han desarrollado algunas variantes de PSOs para codificaciones con variables enteras, en su planteamiento original estas técnicas han sido desarrolladas para problemas con múltiples variables de decisión codificadas mediante vectores de variables reales, motivo por el cual ésta será la única codificación con la que trabajaremos a lo largo de este tema.

Además, es importante destacar que las funciones *Best()* y *Stop()* necesitan conocer el valor de las funciones de mérito del problema sobre los valores de las posiciones de las partículas con los que está trabajando.

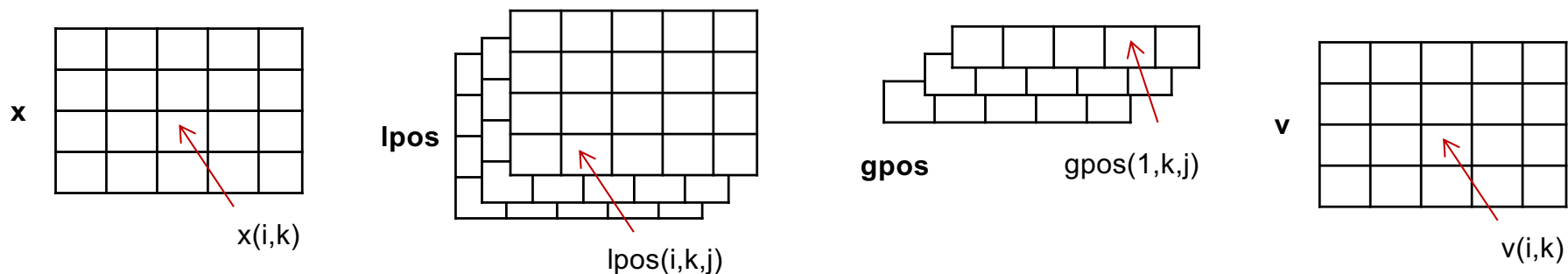
Antes de pasar a describir con detalle la funcionalidad habitual asociada a las funciones *Initialize()*, *Best()*, *Dynamic()* y *Stop()*, vamos a presentar la nomenclatura utilizada para describir las componentes de las variables del PSO, considerando que tenemos un problema de optimización con D variables de decisión reales:

- **Variables de posición:**

- **x** representa la posición de todas las partículas de la nube en el espacio de decisión D.
- **x(i,k)** representa el valor de la k-ésima variable de decisión (D_k) para la partícula i.
- **lpos** representa la mejor posición que el algoritmo ha encontrado para cada partícula de la nube.
- **lpos(i,k,:)** representa el valor (o valores) de la k-ésima variable de decisión (D_k) para la partícula i, almacenado en el mejor local **lpos**.
- **gpos** representa la mejor posición que el algoritmo ha encontrado para todas las partículas de la nube.
- **gpos(1,k,:)** representa el valor (o valores) de la k-ésima variable de decisión (D_k) para todas las partículas, almacenado en el mejor global **gpos**.

- **Variable velocidad:**

- **v** representa la velocidad de todas las partículas de la nube en el espacio de decisión D.
- **v(i,k)** representa el valor de la velocidad de la k-ésima variable de decisión (D_k) para la partícula i.



FUNCIÓN DE INICIALIZACIÓN I

La función *Initialize()* es la **responsable de generar los valores de x de las posiciones y vel de las velocidad de una nube de partículas**. A la hora de generar los valores de las:

- **Posiciones:** sus valores iniciales se encuentran directamente relacionados con el conocimiento que el usuario tenga sobre la disposición de las soluciones del problema de optimización en el espacio de búsqueda:
 - *Si se desconoce* totalmente el valor de las soluciones, lo mas conveniente resulta generar los valores de las posiciones aleatoriamente, siguiendo una distribución uniforme entre el valor máximo y mínimo posible de cada parámetro. De esta forma, se asegura que todos los puntos del espacio son equiprobables inicialmente.
 - *Si se conoce en torno a qué valores* se localizan las solución optima del problema, puede resultar conveniente generar los valores siguiendo una distribución Gaussiana en torno a los mismos. Otra forma de proceder consiste en incluir, entre las soluciones propuestas, las mejores soluciones obtenidas por otros algoritmos de optimización, así como algunas generadas aleatoriamente con una distribución uniforme sobre el espacio de búsqueda.

De hecho, cualquier forma de proceder es válida, aunque se debe tener en cuenta que para facilitar la convergencia del PSO a la solución optima del problema es conveniente evitar que el conjunto de sus soluciones converja a un único valor, para que la función que actualiza las posiciones y velocidades de las partículas sean capaces de trasladarlas a nuevas regiones del espacio. Por lo tanto, la población inicial tiene que ser lo suficientemente diversa para evitar que el PSO converja a un valor no optimo en las generaciones iniciales.

FUNCIÓN DE INICIALIZACIÓN II

- **Velocidades:** sus valores iniciales se encuentran directamente relacionados con el conocimiento que el usuario tenga sobre la disposición de las soluciones del problema de optimización en el espacio de búsqueda:
 - *Si se desconoce* totalmente el valor de las soluciones, lo mas conveniente resulta inicializarlas a un valor nulo, de forma que las tendencias en todas las direcciones sean equiprobables.
 - *Si se conoce en torno a qué valores* se localiza la solución óptima del problema, puede resultar conveniente fijar los valores de las velocidades de forma que las partículas se muevan inicialmente hacia la dirección deseada.

De todas formas, como los valores de las velocidades previas normalmente tienen una influencia mucho mejor en la dinámica de las partículas que las posiciones del mejor individuo local y global, el valor que se elija inicialmente no tendrá una gran influencia, después de unas cuantas iteraciones, en la dinámica seguida por las partículas.

Adicionalmente, y con el objeto de introducir diversidad en la nube actual en cualquier generación, la función de inicialización puede ser utilizada para generar en cada iteración, un pequeño conjunto de partículas, denominadas **inmigrantes**, cuyos valores pueden ser utilizados por la función *Best()* para modificar el valor de las mejores partículas locales y globales.

FUNCIÓN DINÁMICA I – Dinámica básica 1

La función *Dynamic()* actualiza la posición y velocidad de cada partícula teniendo en cuenta su estado (posición y velocidad anterior), una de las mejores posiciones que ha visto a lo largo de la trayectoria para la partícula correspondiente, y una de las mejores posiciones que ha visto a lo largo de la trayectoria de todas las partículas. La forma más básica de proceder se presenta en el pseudo-código de la siguiente función, en el que para cada partícula (de las N totales) y variable de decisión (de las D totales) se actualiza:

1. Su velocidad $v(i,k)$, a partir de una combinación de un porcentaje $w \cdot u_w$ de la velocidad previa, un porcentaje $c_1 \cdot u_l$ de la discrepancia entre la posición de uno de los mejores locales y la posición actual de la partícula, y un porcentaje $c_g \cdot u_g$ de la discrepancia entre la posición de uno de los mejores globales y la posición actual de la partícula. Los valores de las variables u_w , u_l y u_g se generan aleatoriamente, motivo por el que la actualización de la velocidad es un proceso aleatorio. Los mejores seleccionados se eligen de forma aleatoria entre los mejores almacenados.

Función 1: Dinámica básica

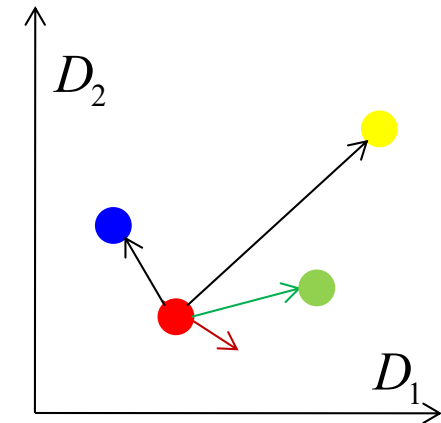
```
[x,v]=DynamicBasic(x,v,lbest,gbest)

1  for i=1:N
2      for k=1:D
3           $u_w=\text{rand}(0,1)$ ,  $u_l=\text{rand}(0,1)$ ,  $u_c=\text{rand}(0,1)$ ,  $s_l=\text{rand}(1,L)$ ,  $s_g=\text{rand}(1,G)$ 
4           $v(i,k)=w \cdot u_w \cdot v(i,k) + c_1 \cdot u_l \cdot (\text{lbest}(i,k,s_l) - x(i,k)) + c_g \cdot u_g \cdot (\text{gbest}(1,k,s_g) - x(i,k))$ 
5           $x(i,k)=x(i,k) + v(i,k)$ 
6      end
7  end
8  return x,v
```


FUNCIÓN DINÁMICA I – Dinámica básica 2

- La posición de la partícula $x(i,k)$, trasladando la partícula desde su posición anterior $x(i,k)$ con el desplazamiento definido por la velocidad actual $v(i,k)$, que ha sido calculada en el paso anterior. Con la forma de proceder elegida, la aleatoriedad en la actualización de la velocidad se transmite a la actualización de la posición.

El funcionamiento básico implementado en la función *DynamicBasic()* se puede analizar de forma gráfica utilizando el esquema de la figura, en la que se representa con un círculo y vector rojo la posición y velocidad actual de la partícula, en azul la posición del mejor local seleccionado, y en amarillo la posición del mejor global seleccionado. La nueva velocidad de la partícula se representa con el vector verde, que se determina como una combinación lineal de los vectores negros y rojo, ponderada en cada dimensión con pesos diferentes, cuyos valores se generan aleatoriamente entre 0 y los máximos w , c_l y c_g . La nueva posición de la partícula, representada con el círculo verde, se obtiene desplazando la posición que tenía (círculo rojo) con su velocidad actual (vector verde).



Los valores máximos w , c_l y c_g determinan, junto con las posiciones y velocidades previas, y las posiciones de los mejores local y global elegidos, el máximo avance permitido por cada partícula. Por lo tanto, sus valores son determinantes para el correcto funcionamiento del algoritmo. Aunque éstos deben ser elegidos según las características del problema (por ejemplo, probando diferentes posibilidades), habitualmente se consideran adecuados:

- $c_l = 2$ y $c_g = 2$.
- w entre 0.3 y 0.5.

FUNCIÓN DINAMICA I – Dinámica básica 3

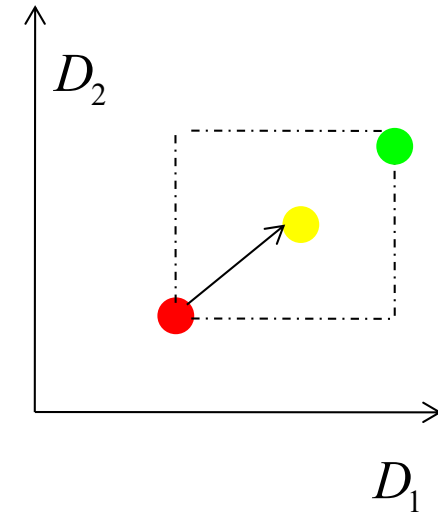
La elección de $c_l = 2$ y $c_g = 2$ permite que las partículas se desplacen de forma simétrica en torno a las posiciones de los mejores para cada una de las componentes de la combinación lineal:

- Si u_l o u_g valen 0, la partícula no se desplaza debido a la posición del mejor correspondiente.
- Si u_l o u_g valen 0.5, la partícula se desplazaría hasta la posición del mejor ($0.5 \cdot 2 = 1$)
- Si u_l o u_g valen 1, la partícula se desplazaría hasta la posición simétrica, respecto al mejor local correspondiente, de la que se encuentra.

La elección de w hace que la velocidad previa se vea reducida al menos un 50%, y que, por lo tanto, el algoritmo se pueda ir frenando en torno a las posiciones de los mejores según se acerca a las mismas y no se localiza nuevas posiciones mejores.

Por último, es importante destacar algunos problemas inherentes de la función dinámica básica:

- Al no existir ningún límite sobre las posiciones de las partículas, durante su evolución pueden terminar localizadas fuera del espacio de búsqueda. Para corregirlo, es conveniente añadir una etapa adicional en la función dinámica que limite los valores de las posiciones.
- Al no existir ningún límite sobre las velocidades de las partículas, es posible que alcancen valores muy elevados, a pesar de los límites en los valores de w , y que las partículas se muevan muy rápido, sin explorar el espacio de forma conveniente y terminando fuera del espacio de búsqueda con relativa facilidad. Para corregir los problemas asociados a las altas velocidades, puede ser conveniente añadir otra etapa en la función dinámica que limite los valores de las velocidades.



FUNCIÓN DINÁMICA II – Límites 1

Para corregir la falta de límites en los valores de las posiciones y velocidades de las partículas, es suficiente con añadir dos etapas adicionales. La nueva versión del algoritmo se presenta a continuación, con la función *LimitPos()* que limita la posición, y la función *LimitSpeed()* que limita la velocidad.

Función 2: Dinámica avanzada

```
[x,v]=DynamicAdvanced(x,v,lbest,gbest)
```

```
1  for i=1:N
2      for k=1:D
3          uw=rand(0,1), u1=rand(0,1), uc=rand(0,1), s1=rand(1,1), s1=rand(1,G)
4          v(i,k)=w*uw*v(i,k)+c1*u1*(lbest(i,k,s1)-x(i,k))+cg*ug*(gbest(1,k,sg)-x(i,k))
5          x(i,k)=x(i,k)+v(i,k)
6      end
7  end
8  [x,v]=LimitPos(x,v)
9  [v]=LimitSpeed(x,v)
10 return x,v
```

A continuación, se presentan las implementaciones más habituales de dichas funciones.

Nótese como en el pseudo-código presentado se observa como la función *LimitPos()* no sólo modifica el valor de la posición, si no también el de la velocidad. Esto es debido a que, si las posiciones están fuera del límite, suele ser conveniente corregir la velocidad para evitar que se siga con la dinámica existente.

FUNCIÓN DINÁMICA I I – Límites 2

Existen diferentes formas de modificar la posición y la velocidad de las partículas cuando la posición de las mismas se encuentra fuera de los límites. Las más habituales, implementadas en la función **LimitPos()**, son las siguientes:

- Posición al límite:** Se fija la posición $x(i,k)$ que se encuentra fuera del límite al valor del límite más cercano. Tras fijar el valor de la posición, es conveniente modificar también el valor de la velocidad correspondiente, habitualmente a través de uno de los dos procedimientos siguientes:

$$x(i,k) = \begin{cases} l_i & \text{si } x(i,k) < l_i \\ u_i & \text{si } x(i,k) > u_i \\ x(i,k) & \text{c.c.} \end{cases}$$
- Velocidad nula:** se frena totalmente la partícula, con el objeto de que su próximo desplazamiento únicamente sea función de la posición de los mejores.

$$v(i,k) = \begin{cases} 0 & \text{si } x(i,k) < l_i \vee x(i,k) > u_i \\ v(i,k) & \text{c.c.} \end{cases}$$
- Velocidad opuesta:** se cambia el signo de la velocidad de la partícula, de forma que esta se comporte como si su velocidad hubiese sido reflejada por las paredes de los límites de las variables de decisión.

$$v(i,k) = \begin{cases} -v(i,k) & \text{si } x(i,k) < l_i \vee x(i,k) > u_i \\ v(i,k) & \text{c.c.} \end{cases}$$
- Choque elástico:** la nueva posición y velocidad de la partícula se determina teniendo en cuenta su posición y velocidad de forma que su comportamiento simule el que se da cuando se produce un choque elástico sobre las paredes definidas por los límites.

$$x(i,k) = \begin{cases} l_i - v(i,k) & \text{si } x(i,k) < l_i \\ u_i - v(i,k) & \text{si } x(i,k) > u_i \\ x(i,k) & \text{c.c.} \end{cases}$$

$$v(i,k) = \begin{cases} -v(i,k) & \text{si } x(i,k) < l_i \vee x(i,k) > u_i \\ v(i,k) & \text{c.c.} \end{cases}$$

FUNCIÓN DINÁMICA I I – Límites 3

Una vez modificada la posición y velocidad de las partículas, teniendo en cuenta los valores impuestos por los límites superior e inferior de cada variable de decisión, puede ser conveniente limitar la velocidad de aquellas partículas que tengan una velocidad demasiado elevada.

La forma mas habitual de proceder, que se implementará en la función **LimitSpeed()**, consiste en determinar aquellas velocidades superiores a la mitad del espacio de búsqueda y modificar los valores de las mismas:

- **Anulándolas**, para que se frenen totalmente
$$v(i,k) = \begin{cases} 0 & \text{si } |v(i,k)| > \frac{u_i - l_i}{2} \\ v(i,k) & \text{c.c.} \end{cases}$$
- **Limitándolas**, para que alcancen el valor del límite impuesto
$$v(i,k) = \begin{cases} \frac{u_i - l_i}{2} & \text{si } v(i,k) > \frac{u_i - l_i}{2} \\ -\frac{u_i - l_i}{2} & \text{si } v(i,k) < -\frac{u_i - l_i}{2} \\ v(i,k) & \text{c.c.} \end{cases}$$

El procedimiento más adecuado depende de las características del problema que se desea resolver. Por lo tanto, si uno no funciona de forma correcta puede ser aconsejable probar otro diferente. Además, como no siempre son necesarios, se puede empezar implementando una versión del PSO sin las funciones *LimitPos()* y *LimitSpeed()*.

FUNCIÓN DE DETERMINACIÓN DE LOS MEJORES I

La función *Best()* del **Algoritmo 2** determina la posición de las partículas que forman parte de los conjuntos de mejores locales (**lbest**) y globales (**gbest**) del PSO.

Aunque los elementos de estos conjuntos corresponden a los valores de las posiciones mejores a lo largo de todas las trayectorias, para seleccionarlos es suficiente con tener en cuenta las nuevas posiciones de las partículas calculadas por el algoritmo, y las posiciones ya almacenadas en los dos conjuntos anteriores.

Otro aspecto a tener en cuenta a la hora de determinar la posición de las partículas mejores es que existen problemas en los que hay un conjunto de posiciones igualmente buenas (como son los problemas monoobjetivo multimodales, o los problemas multiobjetivo, donde se utiliza una función de optimalidad Pareto), motivo por el que suele ser necesario almacenar todas las soluciones igualmente mejores, o por lo menos, un subconjunto significativo de las mismas en **lbest** y **gbest**. Para elegir un subconjunto significativo de valores para **lbest** y **gbest** se pueden utilizar técnicas similares a las que ya hemos visto en otras heurísticas, como son las técnicas de recombinación basada en la “*crowding distance*” o los métodos basados en *clusters*.

Por otra parte, es importante resaltar que existen algunas variantes de PSO en las que en los conjuntos **lbest** y **gbest**, no solo se almacenan las mejores soluciones, si no que también se incluyen los elementos inmediatamente posteriores. Una forma de elegir los elementos, en este caso, consiste en utilizar el **método de torneo** sobre dos elementos elegidos aleatoriamente. De esta forma, la probabilidad de trasladar las partículas hacia las posiciones de los mejores, es mayor que la probabilidad de trasladarlas hacia posiciones peores. Este hecho aumenta la posibilidad de que el algoritmo explore otras regiones del espacio, y puede frenar la convergencia prematura del PSO hacia la región de los mejores locales y globales.

FUNCIÓN DE DETERMINACIÓN DE LOS MEJORES II - l_{best}

A la hora de actualizar los valores del conjunto de mejores locales **l_{best}** dentro de la función *Best()*, para cada una de las partículas se comprueba si los valores de las funciones de mérito asociados a la nueva posición de la partícula hacen que está sea mejor, equivalente, o peor que los valores de las funciones de mérito asociados a cada una de las posiciones almacenadas en **l_{best}** para la misma partícula:

- Si la nueva partícula es mejor, la nueva partícula pasa a ser la única mejor local para dicha partícula.
- Si la nueva es equivalente, se pueden dar dos casos:
 - La nueva pasa a ser la única mejor.
 - La nueva pasa a ser incluida entre las mejores.
- Si es peor, el conjunto de **l_{best}** asociado con dicha partícula no se ve modificado.

Las dos formas de proceder que se proponen en el caso en el que las soluciones sean equivalente, hacen que la actualización de las velocidades y posiciones de la partícula correspondiente en la próxima iteración del algoritmo sea diferente:

- Si la nueva posición pasa a ser la única mejor, el término de desplazamiento correspondiente al local mejor no tiene ninguna influencia, ya que la actual y el mejor local son exactamente el mismo.
- Si la nueva posición pasa a ser incluida entre las mejores de la partícula, el mejor local que se utilice en la actualización puede ser cualquiera de los elementos de **l_{best}** para dicha partícula, y por lo tanto, puede aparecer un desplazamiento hacia cualquier región igualmente buena que haya sido detectada por la partícula a lo largo de su trayectoria.

Por último, para determinar el **l_{best}** en la primera iteración del algoritmo, fuera del bucle, lo único que hay que hacer es igualar sus valores a los de las posiciones iniciales de las partículas.

FUNCIÓN DE DETERMINACIÓN DE LOS MEJORES III - gbest

A la hora de actualizar los valores del conjunto de mejores globales **gbest** dentro de la función *Best()*, se comprueba, para todas las partículas, si los valores de las funciones de mérito asociados a sus nuevas posiciones son mejores, equivalentes, o peores que los valores de las funciones de mérito asociados a cada una de las posiciones almacenadas en **gbest**:

- Si una de las nuevas partículas es mejor, ésta pasa a ser la mejor global, junto con el resto de las nuevas partículas que sean igual de buenas que ella.
- Si ninguna de las nuevas partículas son mejores a las existentes en **gbest**:
 - Las nuevas partículas que son equivalentes pasan a formar parte del **gbest**.
 - Las nuevas partículas peores no participan del conjunto de los mejores globales.

Además, es importante resaltar que cuando el número de elementos en **gbest** es mayor que 1, la selección aleatoria de los elementos de **gbest** para actualizar las velocidades de todas las partículas puede hacer que éstas se desplacen en cada iteración del algoritmo hacia una posición mejor global diferente, sin llegar a acercarse a ninguna de ellas. Para contrarrestar este comportamiento, se puede introducir una función de selección que tenga en cuenta las selecciones pasadas, de forma que se mantenga el elemento elegido durante unas cuantas iteraciones cuando se considere conveniente.

Por último, para determinar el **gbest** en la primera iteración del algoritmo, fuera del bucle, lo único que hay que hacer es escoger los mejores entre las posiciones iniciales de las partículas.

FUNCIÓN DE PARADA

Por último, para terminar la ejecución del PSO, es necesario definir la función de finalización del algoritmo. Las versiones mas sencillas y habituales consisten en:

- **Límite de generaciones:** el algoritmo termina una vez que se alcanza un número de generaciones prefijado.
- **Convergencia de la población:** el algoritmo también puede terminar cuando la diferencia de los valores de la cadena o de las funciones de mérito de la población es menor que un valor prefijado, o cuando las mejores soluciones o sus funciones de mérito no cambia de forma significativa durante un número prefijado de generaciones. La primera forma de proceder mide la convergencia de todas las partículas a una región del espacio, mientras que la segunda mide la convergencia de los mejores hacia un valor determinado.

En cualquier caso, una vez terminada la ejecución del algoritmo, éste devuelve los individuos que se encuentran en el conjunto **gbest**, ya que éste almacena las mejores soluciones calculadas por el algoritmo a lo largo de todas sus iteraciones.

Finalmente, para evitar que se tenga que recalcular los valores de las funciones de mérito para los valores de las posiciones mejores locales (**lbest**) y/o globales (**gbest**) en cada llamada a las funciones *Best()* y *Stop()*, dichos valores pueden ser almacenados en una variable del programa principal, que se utilizará como entrada a ambas funciones, y salida de la primera.

Interpretación física

Aunque originariamente el PSO se fundamenta en el comportamiento social de una bandada de pájaros, el movimiento de las partículas se puede reinterpretar de forma física, considerando que cada parámetro de decisión y partícula describen la trayectoria asociada a la siguiente ecuación diferencial:

$$m\ddot{x} = -b\dot{x} + k_l(x_l^* - x) + k_g(x_g^* - x)$$

que representa a un sistema mecánico de masa m , que se encuentra atraído, por medio de dos funciones potenciales, hacia las posiciones de su mejor local x_l^* y global x_g^* , y frenado por una fuerza de rozamiento asociado a su velocidad $\dot{x}$ a través de b .

La solución de dicha ecuación diferencial, suponiendo que $m=1$, y que la posición y velocidad permanecen constantes durante el intervalo de integración $\Delta t=1$:

$$\begin{aligned}\dot{x}[t] - \dot{x}[t-1] &= (-b\dot{x}[t-1] + k_l(x_l^* - x[t-1]) + k_g(x_g^* - x[t-1]))\Delta t \rightarrow \dot{x}[t] = (1-b)\dot{x}[t-1] + k_l(x_l^* - x[t-1]) + k_g(x_g^* - x[t-1]) \\ x[t] - x[t-1] &= \dot{x}[t]\Delta t \rightarrow x[t] = x[t-1] + \dot{x}[t]\end{aligned}$$

Haciendo $w^*u_w=(1-b)$, $c_lu_l=k_l$ y $c_gu_g=k_g$, se obtienen las ecuaciones dinámicas del PSO. Por lo tanto, las partículas, utilizando esta interpretación física, son frenadas por una fuerza de rozamiento con un coeficiente generado aleatoriamente entre unos valores determinados y atraídas por los potenciales generados por las posiciones mejores locales y globales. Los coeficientes de los potenciales también se calculan aleatoriamente, motivo por el que la posición a la que tiende la partícula es modificada en cada iteración del algoritmo.

Una vez establecida esta analogía, es posible desarrollar nuevas variantes de PSO en las que en vez de ser atraídas las partículas por dos potenciales clásicos, sean atraídas por otros (por ejemplo cuánticos).

Mutaciones

Como la traslación de las partículas depende de su velocidad y de los potenciales atractivos generados por las mejores posiciones locales y globales, en los casos en los que únicamente exista una mejor solución global y que las partículas no detecten nuevas posiciones mejores durante varias generaciones sucesivas, es posible que todas las partículas tiendan a la mejor posición global, y que por lo tanto, no puedan ser mejoradas en generaciones sucesivas.

Para corregir este problema algunas versiones de PSO incluyen una etapa adicional de mutación, en la que alguno de los valores de algunas variables de decisión de algunas partículas es modificado de forma aleatoria. Estas funciones de mutación pueden ser cualquiera de las existentes en los GAs representados con cadenas de variable reales.

Además, en algunos algoritmos se combinan distintos tipos de mutaciones: por ejemplo en OMOPSO, una variante multiobjetivo de PSO, a un tercio de las partículas de la nube se le aplica la mutación uniforme, a otro tercio la mutación no uniforme, y el tercio restante no es modificado por la función de mutación.

Algoritmo 3: PSO con mutación

```
gbest=PSO()  
1  [x,v]=Initialize()  
2  [lbest,gbest]=Best(x)  
3  repeat:  
4      [x,v]=Dynamic(x,v,lbest,gbest)  
5      x=Mutation(x)  
6      [lbest,gbest]=Best(x,lbest,gbest)  
7  until Stop(gbest)  
8  return gbest
```

Optimización de problemas multi-objetivo

Para convertir un PSO básico en una técnica de optimización multiobjetivo es suficiente con almacenar en los **gbest** y **lbest** los conjuntos de posiciones mejores en vez de únicamente la mejor solución, y actualizar la velocidad de cada partícula utilizando un mejor local o global elegido aleatoriamente.

La forma de implementar esta forma de proceder no es única y ha dado lugar a diferentes PSO multiobjetivo, alguno de los cuales se consideran estándares en la actualidad. La forma de proceder de alguna de estas técnicas se detalla en las siguientes publicaciones:

- S. Mostaghim, J. Teich. Covering Pareto-Optimal Fronts by Subswarms in Mutl-objective Particle Swarm Optimization. CEC 2004.
- X. Li. A Non-dominated Sorting Particle Swarm Optimizer for Multiobjective Optimization. EMO 2003.
- D. Srinivasan, T. H. Seow. Particle Swarm Inspired Evolutionary Algorithm (PS-EA) for Multiobjective Optimization Problme. CEC 2003.
- S. Ho, S. Yang. G. Ni, E.W. Lo, H. Wong. A Particles Swarm Optimization-Based Method for Multiobjective Design Optimizations. IEEE Transaction on Magenetics. 2005.
- M. Reyes, C. A. Coello Coello. Improving PSO- bBased Multiobjective Optimization Using Crowding, Mutation and ϵ -dominance. EMO 2005.

La tercera familia de técnicas evolutivas que vamos a presentar en este tema son los algoritmos de **evolución diferencial** (*Differential Evolution*, DE), cuya primera versión fue desarrollada por K. Price y R. Storn en 1995 cuando intentaban resolver el problema del ajuste de los coeficientes de polinomios de Chebicheff a un conjunto de datos. La idea principal del algoritmo consiste en perturbar los individuos de una población utilizando las diferencias de valores existentes entre parejas de individuos de la misma población. Al adaptar las perturbaciones a las **diferencias** de parejas de individuos existentes, las direcciones de búsqueda y el tamaño máximo de los incrementos **evoluciona** de acuerdo con los individuos existentes en la población en cada iteración del algoritmo. Es decir, la población **evoluciona** automáticamente de forma **diferencial**, y por este motivo, esta familia de algoritmos ha sido bautizado como DE.

Como la evolución diferencial, por si misma, no es suficiente para asegurar el buen comportamiento del algoritmo, en el mismo se incluye una etapa de combinación de los valores de las soluciones perturbadas, y otra etapa que selecciona, entre las nuevas propuestas y las antiguas, aquellas que formarán parte de la población en la iteración siguiente. El **Algoritmo 4** muestra el pseudocódigo para la implementación de un DE básico, donde **x** representa a la población (conjunto de soluciones) actual, y **x_new** a las nuevas soluciones perturbadas y recombinadas. La función *Initialize()* es la responsable de generar la población inicial, *Perturbate()* de perturbarlas de forma diferencial, *Cross()* de combinar los valores de las soluciones perturbadas con las existentes, *Recombine()* de elegir, entre las soluciones nuevas y las antiguas, cuales pasan a la siguiente iteración, y *Stop()* determina cuando finaliza la ejecución del algoritmo.

Además, es importante destacar que las funciones *Recombine()* y *Stop()*, y en algunos casos *Perturbate()*, necesitan conocer el valor de las funciones de mérito sobre las soluciones con las que están trabajando, y que aunque *Recombine()* comparte el nombre con la función correspondiente de GA, habitualmente implementa otra funcionalidad.

Algoritmo 4: DE básico

```
x=DE()  
1  x=Initialize()  
2  repeat:  
3      x_new=Perturbate(x)  
4      x_new=Cross(x,x_new)  
5      x=Recombine(x,x_new)  
6  until Stop(x)  
7  return x
```

Los DEs sirven para optimizar problemas con variables de decisión reales, que de forma genérica se representarán, dentro de las diferentes funciones del algoritmo, bien como:

- **$\mathbf{x}$** y **$\mathbf{x_new}$** , para mostrar los valores de conjuntos de soluciones en el espacio de decisión D .
- $x(i,k)$ y $x_new(i,k)$, para mostrar el valor de la k -ésima variable de decisión (D_k) para la i -ésima solución en el conjunto correspondiente **$\mathbf{x}$** o **$\mathbf{x_new}$** .

Las múltiples versiones de DE que se encuentran en la literatura básicamente se diferencian en el comportamiento implementado dentro de las 5 funciones del algoritmo DE básico (representado en el pseudo-código del **Algoritmo 4**):

- *Generate()*: es la función responsable de generar la población inicial del DE. Habitualmente, los valores de las soluciones iniciales son generadas aleatoriamente, aunque también es posible incluir entre los mismos las soluciones obtenidas por cualquier otro algoritmo de optimización.
- *Perturbate()*: es la que perturba las soluciones existentes utilizando las diferencias existentes entre parejas de individuos elegidos aleatoriamente. Las soluciones perturbadas pueden ser elegidas aleatoriamente o teniendo en cuenta la aptitud de los individuos. Además, en algunos casos, la perturbación aplicada tiene también una componente aleatoria.
- *Cross()*: es la que combina la información contenida en las soluciones perturbadas y las ya existentes. El proceso de combinación es aleatorio, y una misma pareja de solución perturbada y existente puede dar lugar a diferentes soluciones.
- *Recombine()*: es la que elige las soluciones de la población para la próxima iteración del algoritmo, teniendo en cuenta su aptitud, es decir, teniendo en cuenta los valores de las funciones de mérito del problema. La función enfrenta cada uno de los nuevos individuos perturbados y cruzados con uno de los individuos existentes, y selecciona el mejor.
- *Stop()*: es la función que determina cuándo se termina la ejecución del DE, teniendo en cuenta los valores de la población.

A lo largo de esta sección, se presentan algunas de las funciones utilizadas habitualmente en los DEs.

FUNCIÓN DE GENERACIÓN:

Esta función es la **responsable de generar los valores de un conjunto de individuos** del algoritmo de evolución diferencial. Su funcionalidad se encuentra directamente relacionada con el conocimiento que el usuario tenga sobre la disposición de las soluciones del problema de optimización en el espacio de búsqueda:

- *Si se desconoce* totalmente el valor de las soluciones, lo mas conveniente resulta generar los valores de las soluciones aleatoriamente, siguiendo una distribución uniforme sobre los valores de la codificación elegida. De esta forma, se asegura que todos los puntos del espacio son inicialmente equiprobables.
- *Si se conoce en torno a qué valores* se localizan las soluciones óptimas del problema, puede resultar conveniente generar los valores de la codificación elegida siguiendo una distribución Gaussiana en torno a los mismos. Otra forma de proceder consiste en incluir, entre las soluciones propuestas, las mejores soluciones obtenidas por otros algoritmos de optimización, así como algunas generadas aleatoriamente con una distribución uniforme sobre el espacio de búsqueda.

De hecho, cualquier forma de proceder es válida, aunque se debe tener en cuenta que para facilitar la convergencia del DE a la solución óptima del problema, es conveniente evitar que el conjunto de sus soluciones converja a un único valor, para que el operador de perturbación sea capaz de generar soluciones diferentes a las existentes. Por lo tanto, la población inicial tiene que ser lo suficientemente diversa para evitar que el DE converja a un valor no óptimo en las generaciones iniciales.

FUNCIÓN DE PERTURBACIÓN I

La función *Perturbate()* selecciona un conjunto de soluciones de la población actual, y perturba sus valores teniendo en cuenta las diferencias que existen entre los valores de parejas de individuos elegidos aleatoriamente. La forma de proceder habitual se muestra en la línea 7 del pseudo-código de la **Función 3**:

el valor de la *k*-ésima variable de decisión para la *i*-ésima solución perturbada se obtiene modificando el valor de una solución *base* con la discrepancia de los valores de las soluciones *d1* y *d2*.

La forma de elegir la solución *base* y las soluciones *d1* y *d2* utilizadas para calcular la discrepancia, así como la forma de obtener el valor de *F* que escala la discrepancia, distingue unas funciones de perturbación de otras.

En cualquiera de los casos, los índices *base*, *d1* y *d2* se obtienen una vez para cada individuo nuevo que se crea, y por lo tanto, cada solución nueva se obtiene teniendo en cuenta la misma base y elementos discrepantes.

Por otra parte, el valor de la *F* se puede obtener en la línea 1 (en ese caso sería el mismo para todas las soluciones), en la línea 4 (en ese caso es la misma para cada solución), o en la línea 6 (en ese caso es diferente para cada parámetro de la solución). En los dos primeros casos, el valor de *F* escala las diferencias existentes en la población, mientras que en el último el valor de *F* escala y rota el valor de dichas diferencias.

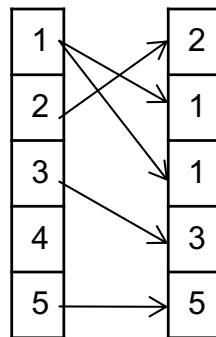
Función 3: Perturbacion

```
x_new=Perturbate(x)
1  F=ObtainF()           %Fixed
2  for i=1:N
3      [base,d1,d2]=SelectIndex(x)
4      F=ObtainF()       %Dither
5      for k=1:D
6          F=ObtainF(); %Jitter
7          x_new(i,k)=x(base,k)+F(x(d1,k)-x(d2,k))
8      end
9  end
10 return x
```

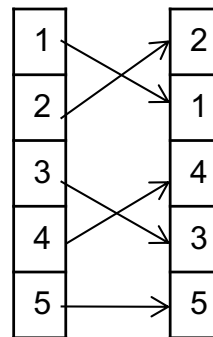

FUNCIÓN DE PERTURBACIÓN II – Selección de la base 1

Para seleccionar el valor del índice de la base se pueden seguir distintos procedimientos:

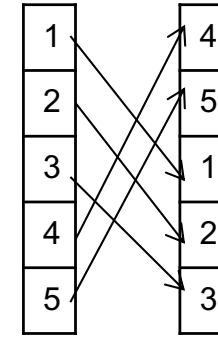
- **Selección aleatoria:** el valor de los índices se elige de forma aleatoria entre los N posibles sin tener en cuenta la aptitud (i.e. los valores de las funciones de mérito) sobre la población de soluciones del problema. Habitualmente, se utiliza alguno de los siguientes métodos:
 - *Uniforme:* se utiliza una distribución uniforme entre 1 y N para elegir, con la misma probabilidad, cualquier individuo de la población.
 - *Basada en perturbación:* para asegurar que cada solución es elegida una única vez, antes del bucle principal de la **Función 3** (línea 2) se genera una perturbación de índices en el rango 1 y N, y el índice de la base para el nuevo elemento i se corresponde al índice que aparece en la posición i de la perturbación generada.
 - *Basada en un offset:* para asegurar que cada solución es elegida una única vez, y que el índice de la solución de la base no coincide con el índice i de la nueva solución, se eligen los valores de forma ordenada desplazados un *offset* obtenido aleatoriamente.



Uniforme



Perturbación



Offset

FUNCIÓN DE PERTURBACIÓN II – Selección de la base 2

- **Selección entre los mejores:** una opción alternativa consiste en elegir como la base el mejor elemento, o en los casos en los que haya un conjunto de individuos igualmente buenos, un individuo de dicho conjunto.

Para reducir la presión que esta forma de proceder ejerce sobre la zona del espacio de búsqueda en la que se encuentran las mejores soluciones, y por lo tanto evitar una convergencia prematura del algoritmo hacia dicha zona, se puede utilizar algún otro método de selección de la base que de mayor probabilidad a los mejores, pero que permita elegir a los peores con una cierta probabilidad. Para alcanzar este fin, se pueden utilizar algunos de los métodos de selección que ya hemos visto en la sección de los algoritmos genéticos, como son el **método de la ruleta**, el **método del torneo**, etc. A la hora de implementar versiones de estos métodos para DEs, es necesario tener en cuenta que, en el caso de los GAs, se obtenían parejas de padres y no los índices de un único individuo, tal y como se requiere para elegir la base en la etapa de perturbación del DE.

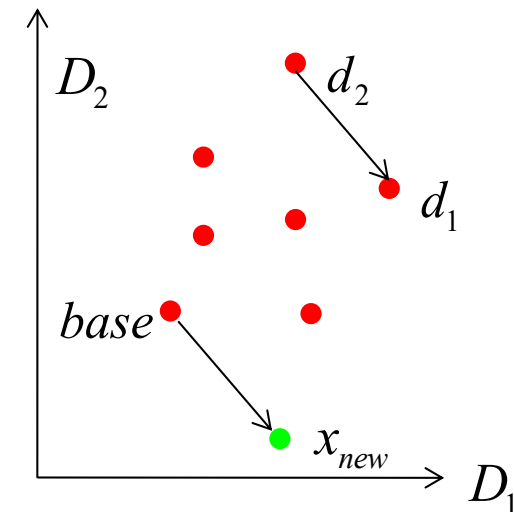
FUNCIÓN DE PERTURBACIÓN III – Selección de los individuos de la perturbación

La elección de los elementos utilizados para realizar la perturbación, y por lo tanto el valor de los índices $d1$ y $d2$, se hace siempre de forma aleatoria entre los índices existentes.

Además, para evitar que la perturbación sea nula (caso que se da cuando $d1$ y $d2$ son iguales), o la coincidencia de cualquiera de los dos índices con el valor del índice de la *base* o el del nuevo individuo i , habitualmente:

- Se elige el valor de $d1$ una vez que ya se conoce el de la base, utilizando una distribución uniforme sobre todos los índices salvo el de la *base* e i .
- A continuación, se elige el valor de $d2$, utilizando una distribución uniforme sobre todos los índices salvo el de la *base*, i y $d1$.

Una vez que se obtienen los valores de los índices de la base y de los elementos de la perturbación, el nuevo elemento $\mathbf{x_new}(i,:)$ se calcula utilizando, como punto de partida la *base*, y como perturbación la discrepancia entre los valores de las soluciones indexadas por $d1$ y $d2$. En la siguiente figura se ilustra la operación de la perturbación para un problema con 2 variables de decisión cuando se usa un valor $F = 1$ fijo para todo individuo.



FUNCIÓN DE PERTURBACIÓN IV – Elección de los valores de F

La elección del valor F con el que se escalan (y en algunos casos rotan) los valores de los desplazamientos de la perturbación, se pueden calcular en, 1) diferentes partes de la función *Pertubate()*, y 2) con diferentes procedimientos:

- **Localización del cálculo de la F :**
 - Fuera del bucle principal, con el objeto de usar la misma F en todos los individuos de la población en cada iteración.
 - Para cada individuo de la población, antes del bucle que itera sobre cada una de las variables de decisión.
 - Para cada individuo y variable de decisión.
- **Obtención del valor de la F :**
 - Utilizar un valor prefijado por el usuario. Una variante consiste en prefijar un valor para cada variable de decisión.
 - Generar un número aleatoriamente utilizando una distribución uniforme entre un valor máximo y mínimo.
 - Generar un numero aleatorio utilizando una distribución Gaussiana con una media y varianza dadas. Habitualmente, si se procede de esta forma, el valor medio se fija a cero, y la varianza se iguala a 1.

Los valores de F del primer caso, o sus límites en el segundo, se eligen dentro del rango $[0,1]$. El valor inferior de 0 a veces se sustituye por el valor F_{crit} , donde p_{cr} es el valor de la probabilidad de cruce asociado a una función *Cross()* del tipo uniforme (su comportamiento se especifica en la siguiente sección).

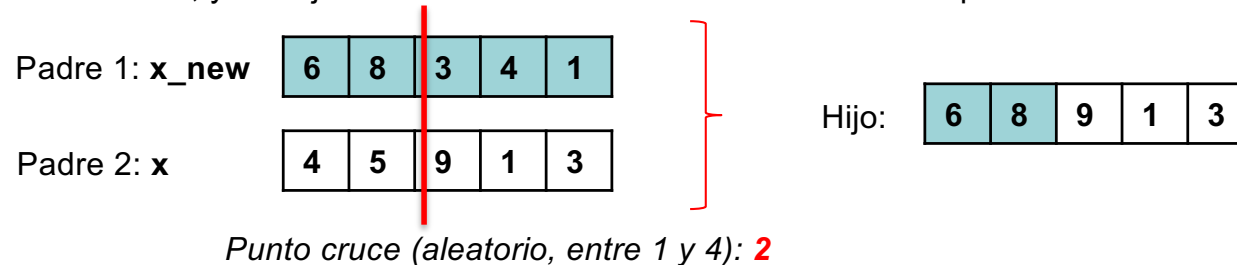
$$F_{crit} = \sqrt{\frac{1-p_{cr}}{N}}$$

FUNCIÓN DE CRUCE I

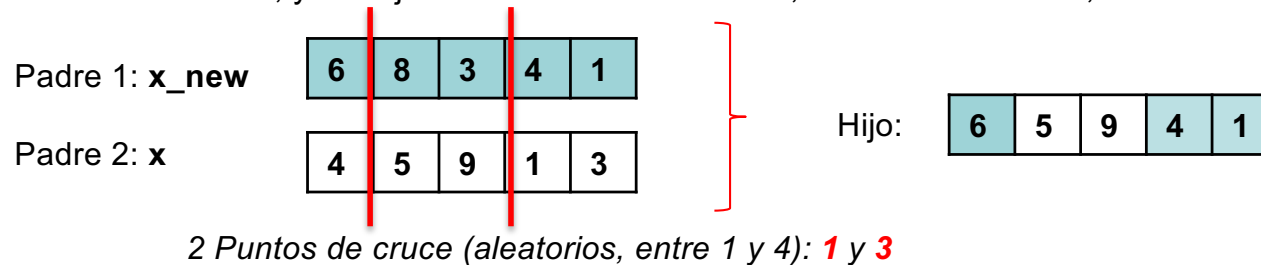
La función de cruce combina la información de los individuos de la población existente $\mathbf{x}$ con los individuos $\mathbf{x_new}$ obtenidos en la operación de perturbación. La combinación se hace por parejas, siguiendo el orden establecido en las dos poblaciones (i.e., el individuo i -ésimo en $\mathbf{x}$ con el individuo i -ésimo en $\mathbf{x_new}$).

Las operaciones de cruce más utilizadas ya las hemos estudiado en la sección de los **algoritmo genéticos**, con la salvedad de que en los GAs, de una pareja de individuos se creaban 2 hijos, mientras que en DE, de la pareja de padres (individuo antiguo e individuo nuevo) solo se crea un hijo. En particular, habitualmente se utilizan:

- **Cruce por 1 punto:** se determina de forma aleatoria, siguiendo una distribución habitualmente uniforme, el punto de cruce de las dos cadenas, y los hijos se crean concatenando la cadena de un padre con la del otro.

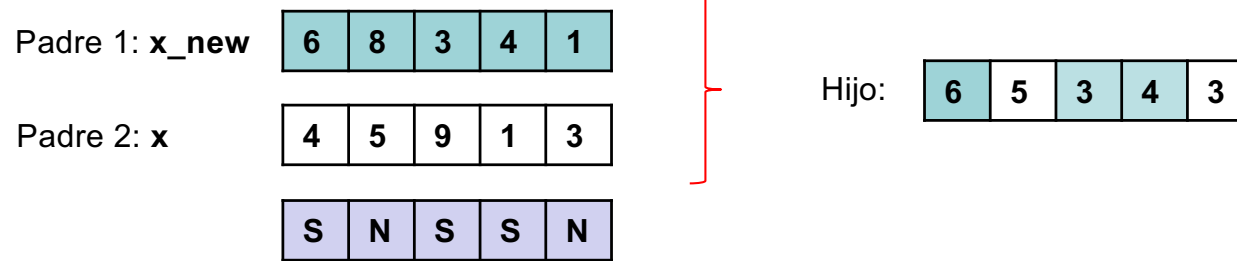


- **Cruce por N puntos:** se determinan de forma aleatoria, siguiendo una distribución habitualmente uniforme, los puntos de cruce de las dos cadenas, y los hijos se crean concatenando, de forma alternada, las cadena de un padre con las del otro.



FUNCIÓN DE CRUCE II

- **Cruce uniforme:** se determina de forma aleatoria, a partir de una probabilidad p_{cr} , si cada elemento de la cadena del primer padre pertenece al primer hijo o no.



Aleatoriamente se determina que los elementos 1, 3 y 4 se toman del mismo padre, y los elementos 2 y 5 del contrario.

De forma general, se considera que un valor de $p_{cr} < 0.2$ es adecuado, como punto de partida, para DE.

FUNCIÓN DE RECOMBINACIÓN

La función de recombinación elige, entre los individuos de la población existente $\mathbf{x}$ y los nuevos $\mathbf{x_new}$ obtenidos a partir de las operaciones de perturbación y mutación, los individuos que formarán parte de la población en la próxima iteración del algoritmo. La selección se hace por parejas, siguiendo el orden establecido en las dos poblaciones. Es decir, el i -ésimo individuo de la población resultante se elige entre el i -ésimo individuo de $\mathbf{x}$, y el i -ésimo individuo de $\mathbf{x_new}$.

En esta comparación por pares ordenados:

- Se elige el mejor de los dos, teniendo en cuenta las funciones de mérito del problema que se desea resolver y la función de optimalidad elegida.
- En el caso en el que los dos sean equivalentes, se elige el individuo de la nueva población.

Esta forma de proceder es muy sencilla, y puede ser aplicada sin ninguna dificultad a problemas monoobjetivo, multiobjetivo, o problemas con restricciones, ya que únicamente es necesario determinar cual es el mejor de cada pareja.

El método de proceder se puede considerar elitista, ya que conserva al menos uno de los mejores individuos encontrados hasta el momento. Sin embargo, es importante tener en cuenta que se pueden perder mejoras intermedias, cuando el nuevo individuo es mejor que buena parte de los individuos de la población existente, pero peor que su pareja correspondiente. Este hecho, no tiene porque ser necesariamente negativo, ya que puede reducir la presión de búsqueda del algoritmo sobre zonas de bondad intermedia.

Por último, en los casos en los que dentro del método de perturbación la elección de la *base* sea aleatoria, la función de selección es la responsable de dirigir totalmente el proceso de búsqueda. Cuando la elección de la base tiene en cuenta la bondad de los individuos, parte de la dirección del proceso de búsqueda se realiza durante el proceso de perturbación.

FUNCIÓN DE PARADA

Por último, para terminar la ejecución del DE, es necesario definir la función de finalización del algoritmo. Las versiones mas sencillas y habituales consisten en:

- **Límite de generaciones:** el algoritmo termina una vez que se alcanza un número de generaciones prefijado.
- **Convergencia de la población:** el algoritmo también puede terminar cuando la diferencia de los valores de la cadena, o de las funciones de mérito de la población, es menor que un valor prefijado, o cuando las mejores soluciones, o sus funciones de mérito, no cambia de forma significativa durante un número prefijado de generaciones. La primera forma de proceder mide la convergencia de todas las soluciones a una región del espacio, mientras que la segunda mide la convergencia de los mejores hacia un valor determinado.

Finalmente, para evitar que se tenga que recalcular los valores de las funciones de mérito para los valores de $\mathbf{x}$ en cada llamada a las funciones *Recombine()*, *Stop()*, y opcionalmente *Perturbate()*, dichos valores pueden ser almacenados en una variable del programa principal, que se utilizará como entrada a dichas funciones, y como salida de la primera.

Perturbaciones con varias parejas de individuos

En la perturbación que hemos visto en la sección 4.2 el desplazamiento del individuo elegido como base se realizaba teniendo en cuenta la discrepancia existente entre una pareja de individuos elegidos aleatoriamente.

Para permitir mas desplazamientos que los soportados por este método, es posible perturbar una base con los desplazamientos producidos por varias parejas.

Para poder llevarlo a cabo, la función *Perturbate()* necesita obtener un par de índices y un factor de escalado (y rotación) diferente para cada pareja de padres que se utilice en la perturbación de la base.

El pseudo-código de la función de la derecha muestra una nueva versión de la función *Perturbate()* que esquematiza los cambios necesarios.

La perturbación con múltiples parejas permite explorar, en el caso en el que las *Fs* se obtengan aleatoriamente, el espacio definido por la combinación lineal de los incrementos definidos por cada pareja. Por lo tanto, permite explorar una zona mas amplia del espacio por medio de una única perturbación.

Sin embargo, como la perturbación utilizando múltiples parejas de variables es computacionalmente más costosa que la perturbación básica, es conveniente probar inicialmente el método básico, y únicamente utilizar el método presentado en esta transparencia cuando el básico no nos permita obtener una solución lo suficientemente buena en un tiempo adecuado.

Función 4: Perturbacion multiples parejas

```
x_new=Perturbate(x)
```

```
1  [F1,..Fp]=ObtainF()           %Fixed
2  for i=1:N
3      [base,d1,d2,d3,...,d2p]=SelectIndex(x)
4      [F1,..Fp]=ObtainF()       %Dither
5      for k=1:D
6          [F1,..Fp]=ObtainF(); %Jitter
7          x_new(i,k)=x(base,k)+ΣFi(x(di,k)-x(di+p,k))
8      end
9  end
10 return x
```

Tratamiento de soluciones no factibles

El algoritmo DE básico presentado hasta el momento no comprueba si los valores de las variables de decisión se encuentran dentro de los límites máximo y mínimo habituales. Para corregir este comportamiento se suele incluir en la implementación final del DE una etapa adicional (denominado *InRange()* en el pseudo-código de la derecha) entre las etapas de cruce y selección del DE básico que modifica aquellos valores que están fuera del rango válido, habitualmente fijando su valor al del límite mas cercano.

Utilización de Archivos auxiliares para evitar la perdida de soluciones optimas equivalentes

La función de recombinación *Recombine()*, que únicamente compara la bondad de parejas de soluciones, desechará una solución optima cuando esta se enfrente a una solución igualmente buena. Este hecho, positivo desde el punto de vista de la optimización, puede dificultar una buena determinación de las soluciones optimas equivalentes en los casos en los que muchas soluciones optima equivalentes se comparen entre si.

Una forma sencilla de resolver el problema consiste en utilizar una variable archivo auxiliar, cuyo valor se actualiza antes de la función *Recombine()* de forma que almacene todas las soluciones igualmente buenas, ya existentes o nuevas. Para limitar el tamaño de la memoria necesaria para el archivo auxiliar se pueden utilizar algunas de las técnicas de muestreo del frente Pareto que hemos visto en las secciones relacionadas con los algoritmos genéticos y las nubes de partículas.

Algoritmo 5: DE limitado

```
x=DE()  
1  x=Initialize()  
2  repeat:  
3      x_new=Pertubate(x)  
4      x_new=Cross(x,x_new)  
5      x_new=InRange(x_new)  
6      x=Recombine(x,x_new)  
6  until Stop(x)  
7  return x
```

Nomenclatura de especificación habitual de los algoritmos de DE

Para especificar de forma compacta el tipo de funciones de perturbación y cruce utilizados en un DE se utiliza habitualmente la siguiente nomenclatura: **DE/SelectionBase/#ParejasPertubacion/TipodeCruce**, donde:

- La primera etiqueta identifica el algoritmo como uno de la familia de DE.
- La segunda etiqueta como se selecciona la base:
 - rand: de forma aleatoria
 - best: entre los mejores
- La tercera etiqueta identifica cuantas parejas de padres se utilizan en la fase de perturbación (y por lo tanto habitualmente toma el valor de 1).
- La cuarta el tipo de cruce utilizado:
 - bin: cruce uniforme (binomial)
 - 1: cruce por un punto

Por ejemplo DE/rand/1/bin, que constituye la primera versión propuesta por K. Price y R. Storn , es un DE con selección aleatoria de la base, 1 pareja de padres en la perturbación, y cruce uniforme.

Optimización de problemas multi-objetivo

Para convertir un DE básico en una técnica de optimización multiobjetivo no es necesario hacer grandes cambio, salvo que se desee añadir un archivo auxiliar que almacene el Frente, ya que a la hora de elegir los mejores durante la comparación de pares lo único que hay que hacer es utilizar las funciones de merito y definición de optimo.

Sin embargo, en la literatura se pueden encontrar diferentes variantes, en las que habitualmente se modifica el método de recombinación para aproximarlos a las técnicas existentes en los algoritmos genéticos (en las que todos los individuos actuales y nuevos compiten entre si), tal y como se pone de manifiesto en las técnicas presentadas en las siguientes publicaciones:

- N. K. Madavan. Multiobjective Optimization using a Pareto Differential Evolution Approach. CEC 2002.
- A.W. Ioro, X. Li. Solving rotated multi-objective optimization problems using differential evolution. AI 2004.
- S. Kukkonen, J. Lampinen. An Extension of Generalized Differential Evolution for Multi-Objective Optimization with Constraints. PPSN VIII. 2004.
- T. Robic, B. Filipic. DEMO: Differential Evolution for Multiobjective Optimization. EMO 2005. 2005.
- L.V. Santana-Quintero, C.A. Coello Coello. An Algorithm Based on Differential Evolution for Multi-Objective Problems. International Journal of Computational Intelligence Research. 2005

5 COMPARATIVA

En este tema hemos visto tres familias de metaheurísticas (algoritmos genéticos, nubes de partículas y evolución diferencial), que se caracterizan todas ellas por buscar los óptimos de un problema evolucionando de forma simultánea los valores de un conjunto de soluciones, teniendo en cuenta los valores de las soluciones previas y las funciones de mérito y criterio de optimalidad elegidos.

Las diferencias principales se encuentran en:

1. Los mecanismos utilizados para crear las nuevas soluciones:

- Los GAs crean las nuevas soluciones por medio de dos operaciones que cruzan (combinan) y mutan (modifican) parte de la información proveniente de las soluciones existentes en la población anterior, seleccionados en base a su aptitud. Es decir, las nuevas soluciones combinan la información asociada habitualmente a buenas soluciones con otra introducida, de forma puramente aleatoria, por el proceso de mutación.
- Los PSOs crean nuevas soluciones desplazando las existentes hacia, 1) las mejores soluciones obtenidas por toda la población, 2) la mejor que cada individuo ha encontrado, 3) el sentido y dirección seguido en la etapa anterior. Por lo tanto, combinan las soluciones actuales, buenas o malas, con la mejor historia pasada (global y local).
- Los DEs crean las nuevas soluciones: 1) perturbando habitualmente todas las soluciones existentes teniendo en cuenta la distancia entre dos soluciones existentes cualquiera, y 2) combinando el valor obtenido con una de las existentes. Es decir, modifican las soluciones existentes teniendo en cuenta la dispersión e información contenida en la población actual, sin tener en cuenta en las formulaciones más básicas la aptitud de las soluciones.

El resultado de los procesos de los tres algoritmos pueden ser soluciones mejores, equivalentes y peores. Además, el uso de la aptitud en los GAs y PSOs durante el proceso de creación no tiene por qué generar soluciones mejores.

5 COMPARATIVA

2. Los mecanismos utilizados para decidir que soluciones forman parte de la nueva generación:

- Aunque los GAs permiten el uso de diferentes mecanismos para llevar a cabo esta etapa de decisión, los que predominan son las técnicas de sustitución elitistas, en las que todos los individuos de la población actual y todos los individuos nuevos compiten entre si, con el objeto de asegurar que las mejores soluciones creadas en todo momento son las que sobreviven. Para evitar problemas de crecimiento de la población en los problemas con múltiples soluciones equivalentes, se utilizan mecanismos que muestrean el frente de soluciones equivalentes, para asegurar que las seleccionadas sean representativas de los mismos.
- Los PSOs mantienen por separado varios conjuntos de soluciones: 1) la nube en la que todas las soluciones nuevas son utilizadas como punto de partida de la próxima iteración del algoritmo, 2) el conjunto de los mejores globales en los que las mejores soluciones alcanzadas por todas las partículas son almacenadas, y 3) el conjunto de los mejores locales de cada partícula donde se almacena la mejor solución obtenida por cada solución de la nube durante todo el proceso. Es decir, esta familia de algoritmos separa las soluciones sobre las que trabaja (nube) de la historia pasada (mejores globales y locales).
- Por último, los DEs comparan cada solución recientemente creada con una solución existente en la población anterior, haciendo que sobreviva la mejor de cada comparación. Esta forma de proceder hace que las soluciones mejores a las de una parte del conjunto de la población se pierdan si éstas son enfrentadas contra una mejor que ellas, por lo que el mecanismo elegido no asegura la supervivencia de las mejores del conjunto, si no de una muestra de las mismas.

Por lo tanto, por una parte los GAs elitistas pueden converger más rápido a una zona del espacio, al eliminar rápidamente soluciones que colectivamente no son muy buenas. Por la otra, los PSOs y DEs convergen más lentamente, al no tomar como punto de partida los mejores de todos los existentes.

5 COMPARATIVA

3. El tipo de codificación sobre la que trabajan

- Los GAs, que fueron originariamente desarrollados sobre codificaciones binarias, utilizan en la actualidad codificaciones binarias, enteras y reales. Además, favorecen la creación de codificaciones naturales como la de las perturbaciones, ya que la idea del cruce y mutación puede ser fácilmente extendida a otras codificaciones.
- Aunque los PSOs fueron originariamente desarrollados sobre codificaciones reales, ya existen algunas versiones especiales para codificaciones enteras. La extensión a codificaciones naturales no es directa, y por lo tanto, son difíciles de modificar para incluir características propias de los problemas con los que se quieren trabajar.
- Por último, los DEs trabajan únicamente sobre codificación reales.

Por lo tanto, desde el punto de vista de las codificaciones, los GAs son en este momento la herramienta más versátil de las tres vistas a lo largo de este tema. Teniendo en cuenta que los DEs y PSOs son técnicas con bastantes menos años de desarrollo, es posible que vayan apareciendo nuevas variantes que incorporen nuevas codificaciones.

4. Complejidad Computacional: los PSOs y DEs son habitualmente técnicas computacionalmente más sencillas que los GAs, y por lo tanto, sus versiones más básicas suelen ser mas fáciles de implementar para construir un prototipo rápido con el que intentar resolver el problema de optimización deseado. Las versiones más elaboradas de todas ellas tienen complejidades similares, ya que muchas nuevas versiones de PSOs y DEs incorporan funcionalidades originales de los GAs, perdiendo de esta forma su simplicidad original.

En cualquier caso, no es sencillo establecer qué técnica será mejor a la hora de resolver por primera vez un problema de optimización. Por lo tanto, en muchos casos la elección se realiza teniendo en cuenta la experiencia que se tiene sobre cada una de las técnicas, ya que ésta facilitará la adaptación de los algoritmos básicos a las necesidades del problema.

TRABAJO PRÁCTICO A ENTREGAR

La entrega correspondiente a este tema consiste en la implementación de alguna de las metaheurísticas basadas en búsquedas evolutivas aquí estudiadas para resolver el problema planteado por el alumno en el Tema 1. Además, en esta entrega se compararán los resultados obtenidos por la técnica utilizada en el Tema 2 con la técnica implementada en esta entrega.

La entrega se ha dividido en dos partes:

- **7 de Junio:** Propuesta con el planteamiento del problema (representación de las soluciones, operadores, funciones objetivo, etc.), y justificación del algoritmo seleccionado, para que los profesores de la asignatura la analicen, y comenten con los alumnos.
- **21 de Junio:** Presentación de un guión, en el que se recoja la implementación final del algoritmo, los resultados obtenidos y una discusión sobre los mismos. En este guión será necesario incluir:
 - Una descripción del funcionamiento de los operadores seleccionados.
 - Una explicación del método utilizado para evaluar las soluciones (coste + tratamiento de las restricciones).
 - Los valores a los que se han fijado los parámetros del algoritmo escogido.
 - Una discusión de los resultados obtenidos, en la que se debe poner de manifiesto:
 - si se mejoran los puntos de partida del algoritmo y si se cumplen las restricciones.
 - si el algoritmo evolutivo mejora las soluciones obtenidas con el algoritmo de búsqueda local utilizado en el tema anterior.

Finalmente, recordar que las sesiones de dudas del tema se realizarán en la semana del **18 al 22 de Mayo**.